# ANALITIK RISIKO KETERLAMBATAN PENGIRIMAN PADA RANTAI PASOK MENGGUNAKAN *MACHINE LEARNING* DAN *GRAPH ANALYTICS*

Studi Kasus: DataCo Smart Supply Chain - Assignment II


Nama: Rafael Yudianto

NIM: 2702752495

Kelas: LCD3


Mata Kuliah: COMP8035041 *Big Data Analytics*


**PROGRAM STUDI MAGISTER TEKNIK INFORMATIKA**

**BINUS GRADUATE PROGRAM**

# DAFTAR ISI

**DAFTAR GAMBAR** ..... **3**
**DAFTAR TABEL** ..... **4**
**BAB 1. PENDAHULUAN** ..... **5**
1.1 Latar Belakang Masalah ..... 5
1.2 Market dan Business Drivers ..... 6
1.3 Pemangku Kepentingan yang Terlibat ..... 6
**BAB 2. RANCANGAN SOLUSI BIG DATA** ..... **7**
2.1 Metode Pengumpulan Data (Data Ingestion) ..... 7
2.2 Teknologi Penyimpanan Data ..... 7
2.3 Processing Framework ..... 8
2.4 Repository Kode dan Artefak Implementasi ..... 9
**BAB 3. TATA KELOLA IT** ..... **10**
3.1 Security ..... 10
3.2 Compliance ..... 10
3.3 Privasi Data ..... 10
3.4 Kualitas Data ..... 10
**BAB 4. DATA** ..... **12**
4.1 Deskripsi Data ..... 12
4.1.1 Pembersihan Data Transaksi ..... 12
4.1.2 Organisasi Data ..... 14
4.2 Model Data dan Operasi Data ..... 14
4.3 Data Tingkat Pesanan untuk Diolah Menggunakan Machine Learning ..... 15
**BAB 5. MACHINE LEARNING** ..... **17**
5.1 Data Preprocessing ..... 17
5.2 Feature Engineering ..... 18
5.3 Algoritma ..... 18
5.4 Proses Pengembangan Model dengan dan tanpa Spark MLlib ..... 19
5.5 Evaluasi Model ..... 20
5.6 Analisis ..... 21
**BAB 6. GRAPH ANALYTICS** ..... **25**
6.1 Pengembangan Graph ..... 25
6.2 Analisis dan Insight yang Diperoleh ..... 26
**BAB 7. VISUALISASI DASHBOARD** ..... **31**
7.1 Proses Membangun Dashboard ..... 31
7.2 Interpretasi Bisnis atau Insights yang Diperoleh dari Dashboard ..... 32
**DAFTAR PUSTAKA** ..... **35**
**BINUS UNIVERSITY ARTIFICIAL INTELLIGENCE USE DECLARATION FORM** ..... **36**

# DAFTAR GAMBAR

Gambar 2.1 Arsitektur dan Alur Pemrosesan DataCo pada Assignment II.....9
Gambar 5.1 Kurva Precision-Recall dan Kurva ROC pada data test: scikit-learn vs Spark MLlib.....21
Gambar 6.1 Subset graf pesanan dan kategori: 210 simpul dan 495 garis penghubung.....27
Gambar 7.1 Antarmuka Utuh Dashboard dan Panel Pemeringkatan Risiko Machine Learning (Top-K Alert).....32
Gambar 7.2 Panel Analisis Topologi Jaringan Kategori Produk dan Kalibrasi Janji Pengiriman (SLA).....33

# DAFTAR TABEL

Tabel 1.1 Market dan business drivers....6
Tabel 1.2 Pemangku kepentingan....6
Tabel 2.1 Implementasi dan batas pengujian komponen....7
Tabel 2.2 Rancangan dan implementasi penyimpanan....8
Tabel 3.1 Pemeriksaan kualitas data dan keterbatasannya....11
Tabel 4.1 Sumber data....12
Tabel 4.2 Pemetaan nama kolom....13
Tabel 4.3 Tahapan pembersihan transaksi....13
Tabel 4.4 Organisasi data....14
Tabel 4.5 Peran variabel....15
Tabel 4.6 Model dan operasi data....15
Tabel 4.7 Cakupan data tingkat pesanan Assignment II....16
Tabel 5.1 Fitur dan representasi masukan model....18
Tabel 5.2 Parameter Logistic Regression....19
Tabel 5.3 Pembagian temporal data pesanan....19
Tabel 5.4 Konfigurasi final dua implementasi Logistic Regression....20
Tabel 5.5 Hasil evaluasi pada data test....20
Tabel 5.6 Hasil evaluasi pada data validation....21
Tabel 5.7 Ringkasan pengujian top-k pada data test....22
Tabel 5.8 Distribusi skor peluang: test dan validation pada eksekusi Google Colab....23
Tabel 5.9 Hasil pengujian ablasi kelompok fitur pada data test....23
Tabel 6.1 Definisi graf pesanan dan kategori....25
Tabel 6.2 Jumlah hubungan sepuluh kategori pada graf lengkap....26
Tabel 6.3 Komponen terhubung pada graf lengkap....26
Tabel 6.4 Tingkat keterlambatan per kategori pada graf lengkap....29

# BAB 1. PENDAHULUAN

## 1.1 Latar Belakang Masalah

Dinamika perdagangan elektronik (*e-commerce*) global dan integrasi rantai pasok modern menuntut tingkat keandalan pemenuhan pesanan (*order fulfillment*) yang semakin tinggi. Dalam lanskap bisnis yang sangat kompetitif, kepuasan konsumen (*customer satisfaction*), loyalitas merek, dan tingkat retensi pelanggan sangat bergantung pada kepastian serta ketepatan waktu pengiriman barang sesuai jadwal yang dijanjikan. Ekspektasi konsumen terhadap transparansi estimasi kedatangan barang dan pelacakan status pesanan secara waktu-nyata kini menjadi standar baku operasional logistik modern.

Keterlambatan pengiriman (*late delivery*) menjadi risiko operasional utama dalam manajemen rantai pasok. Ketika pesanan gagal diserahkan tepat waktu, dampak negatif yang timbul tidak hanya berupa lonjakan beban layanan pelanggan (*customer service*) dan keluhan konsumen. Kegagalan tersebut juga menimbulkan penalti finansial, biaya pengiriman ulang, serta risiko pelanggan beralih ke penyedia lain. Penelitian empiris dalam literatur manajemen operasi dan rantai pasok menunjukkan bahwa deviasi durasi pemrosesan dan keterlambatan pengiriman berkorelasi langsung dengan penurunan skor ulasan pelanggan serta persepsi kualitas layanan secara menyeluruh (Akturk et al., 2022).

Tantangan utama yang dihadapi industri logistik berskala besar adalah karakteristik data operasional yang tergolong sebagai *Big Data*. Rantai pasok modern memproses puluhan ribu transaksi harian yang mencakup jutaan item barang dan ratusan kategori produk. Transaksi tersebut menjangkau wilayah geografis yang luas dengan beragam moda pengiriman serta *service-level agreement* (SLA) yang bervariasi. Sebagian besar sistem logistik konvensional masih mengevaluasi keterlambatan secara reaktif, yaitu mendeteksi masalah setelah jadwal terlewati atau barang telah dibawa kurir. Pendekatan reaktif ini membuat tim operasional kehilangan momentum untuk melakukan intervensi preventif sebelum pesanan dikirimkan.

Untuk beralih dari mitigasi risiko reaktif menuju manajemen risiko proaktif (*proactive risk management*), diperlukan penerapan kerangka kerja *Big Data Analytics* dan *Machine Learning*. Dengan memanfaatkan kombinasi fitur yang tersedia saat pesanan siap diproses, seperti janji durasi pengiriman terjadwal, segmen pasar pelanggan, wilayah pemesanan, dan komposisi keranjang belanja, model prediktif dapat mengestimasi probabilitas risiko keterlambatan sejak dini. Pendekatan ini memungkinkan tim logistik menyusun daftar prioritas penanganan pesanan (*risk prioritization ranking*) sehingga pesanan dengan risiko keterlambatan tertinggi memperoleh alokasi sumber daya pemenuhan lebih cepat.

Selain pendekatan pemodelan tabular, pemahaman struktural mengenai bagaimana produk dipesan secara bersamaan memerlukan perspektif relasional berbasis jaringan. *Graph Analytics* menawarkan kemampuan analitik untuk memodelkan interaksi antara nomor pesanan dan kategori produk sebagai graf bipartit (*bipartite network*). Melalui pemetaan topologi jaringan ini, analisis sentralitas dan keterhubungan kelompok dapat mengungkap pola ko-keanggotaan kategori produk dalam pesanan pelanggan sehingga memperjelas keterkaitan relasional antar-kategori yang kerap dipesan bersamaan.

Berdasarkan urgensi bisnis dan kebutuhan analitik tersebut, penelitian ini memanfaatkan *dataset DataCo Smart Supply Chain* versi 5 (Constante et al., 2019). *Dataset* ini dipublikasikan secara resmi melalui Mendeley Data dengan lisensi Creative Commons Attribution 4.0 International (CC BY 4.0). Penelitian ini bertujuan merancang, mengimplementasikan, dan mengevaluasi solusi analitik *Big Data* secara terpadu pada lingkungan Google Colab. Cakupan penelitian meliputi pembersihan dan agregasi data transaksi dengan PySpark, pengembangan model *Logistic Regression* menggunakan scikit-learn dan *Spark MLlib*, pelaksanaan studi ablasi fitur empiris, serta eksplorasi topologi graf pesanan-kategori menggunakan *Graph Analytics*.

## 1.2 *Market* dan *Business Drivers*

*Market drivers* mencakup ekspektasi pelanggan terhadap estimasi dan pelacakan barang, serta persaingan janji waktu pengiriman. *Business drivers* mencakup beban layanan pelanggan, perencanaan kapasitas gudang, dan pemeliharaan tingkat layanan. Keluaran yang diusulkan berupa daftar prioritas internal, sedangkan manfaat penurunan keterlambatan atau penghematan biaya belum diuji secara langsung. Faktor-faktor tersebut merupakan asumsi kebutuhan bisnis untuk merumuskan masalah, bukan simpulan yang ditarik dari *dataset*. Pengukuran *dataset* hanya menunjukkan distribusi variabel yang tercatat, sedangkan nilai ekonomis dari prediksi risiko tetap merupakan usulan kebutuhan logistik.

**Tabel 1.1 *Market* dan *business drivers***

| Jenis | Pendorong | Kebutuhan analitik |
|---|---|---|
| *Market* | Ekspektasi estimasi dan pelacakan; persaingan janji pengiriman | Informasi risiko lebih awal untuk membantu pengelolaan ekspektasi |
| *Business* | Beban layanan pelanggan dan operasional; kapasitas gudang; tingkat layanan | Daftar prioritas pesanan pada siklus operasional yang dipilih |

## 1.3 Pemangku Kepentingan yang Terlibat

Operations/Logistics merupakan calon pengguna utama skor risiko untuk meninjau prioritas penanganan pesanan. CS Lead (koordinator layanan pelanggan) dapat memakai informasi risiko untuk mendukung komunikasi dengan pelanggan, sedangkan manajemen memerlukan ringkasan layanan. Data/IT serta fungsi keamanan dan privasi mendukung pengelolaan sistem. Peran tersebut merupakan rancangan pemanfaatan keluaran, bukan penerapan operasional yang telah dievaluasi.

**Tabel 1.2 Pemangku kepentingan**

| Pihak | Informasi | Tindakan |
|---|---|---|
| Operations / Logistics | Daftar pesanan dan skor risiko | Meninjau prioritas penanganan |
| CS Lead | Pesanan berisiko | Mengelola komunikasi dan ekspektasi |
| Manajemen | Ringkasan wilayah, kategori, moda | Meninjau tingkat layanan |
| Data/IT | Status *pipeline* dan validasi | Menangani kegagalan pengolahan |
| Keamanan / privasi | Hak akses dan audit | Mengawasi penggunaan data |

# BAB 2. RANCANGAN SOLUSI BIG DATA

Arsitektur pada Assignment I menempatkan Apache Hadoop sebagai ekosistem pendukung, Hadoop Distributed File System (HDFS) sebagai penyimpanan, dan Apache Spark sebagai mesin pemrosesan. HDFS adalah sistem penyimpanan berkas terdistribusi yang membagi data ke beberapa komputer dan menjadi bagian dari ekosistem Hadoop, sementara MapReduce tidak digunakan dalam penelitian ini. Implementasi eksperimen membaca dan menulis data pada *filesystem runtime* Google Colab (file:///). Pengujian HDFS hanya mencakup penulisan dan pembacaan berkas sederhana, bukan pengolahan *dataset* lengkap.

## 2.1 Metode Pengumpulan Data (*Data Ingestion*)

Ketiga CSV DataCo versi 5 diunduh dari sumber resmi. Data historis dibaca secara *batch*, yaitu diproses sebagai kumpulan data utuh, menggunakan Python. *Ingestion* transaksi harian dan *streaming clickstream* tidak dijalankan pada tahap pelaksanaan yang dilaporkan.

Pengolahan memakai *snapshot* historis, bukan *feed* transaksi operasional. Tidak ada klaim kesegaran *real-time*, pembaruan harian produksi, atau respons per detik dalam hasil proyek ini.

Pemeriksaan *clickstream* memperoleh rentang 1 September 2017 pukul 06:00 hingga 31 Januari 2018 pukul 23:58. Struktur, nilai kosong, dan kesamaan baris telah diperiksa. *Clickstream* belum digabungkan dengan data transaksi sehingga manfaat prediktif maupun hubungan sebab-akibat tidak diklaim sebagai hasil penelitian.

**Tabel 2.1 Implementasi dan batas pengujian komponen**

| Fungsi | Implementasi | Belum dilaksanakan |
|---|---|---|
| *Ingestion* | Unduh *snapshot* CSV; verifikasi hash; baca *batch* Python Colab | Pemuatan seluruh CSV ke HDFS; *batch* terjadwal |
| *Storage* | HDFS *single-node*; tulis/baca berkas uji | Lapisan *dataset* lengkap; konversi Parquet/JSON |
| *Processing* | Pembersihan Python; pembentukan *edge* graf dengan Spark; pelatihan dan evaluasi pada Colab | *Streaming* dan pengukuran *multi-node* belum dilakukan |
| *Analytics*/*consumption* | Probabilitas, prediksi, metrik, dan graf tersimpan pada *runtime* Colab | Penggunaan operasional dan intervensi tidak diuji |

## 2.2 Teknologi Penyimpanan Data

Rancangan penyimpanan memisahkan data mentah, data bersih, dan data siap analitik dalam lapisan yang dapat dibaca Spark. HDFS menjadi pilihan penyimpanan pada rancangan tersebut. Dalam implementasi, ketiga lapisan diwakili oleh direktori dan berkas pada *runtime* Google Colab. Pengujian berkas sederhana pada HDFS menunjukkan fungsi tulis dan baca dasar, tetapi *dataset* penelitian tidak dimuat lengkap ke HDFS.

HDFS memakai NameNode untuk mengelola metadata dan DataNode untuk menyimpan blok data. Konfigurasi uji pada Google Colab memakai satu DataNode dengan dfs.replication=1. Konfigurasi ini tidak menyediakan replikasi lintas mesin sehingga toleransi kegagalan simpul, ketersediaan tinggi, dan skalabilitas produksi tidak diuji. Data historis disimpan sebagai berkas CSV (comma-separated values) untuk pengolahan *batch*. Format Parquet untuk transaksi bersih dan JSON (JavaScript Object Notation) untuk *clickstream* merupakan format dalam rancangan, bukan hasil konversi yang telah digunakan. Penjelasan arsitektur HDFS mengacu pada dokumentasi Apache Software Foundation (n.d.-a).

**Tabel 2.2 Rancangan dan implementasi penyimpanan**

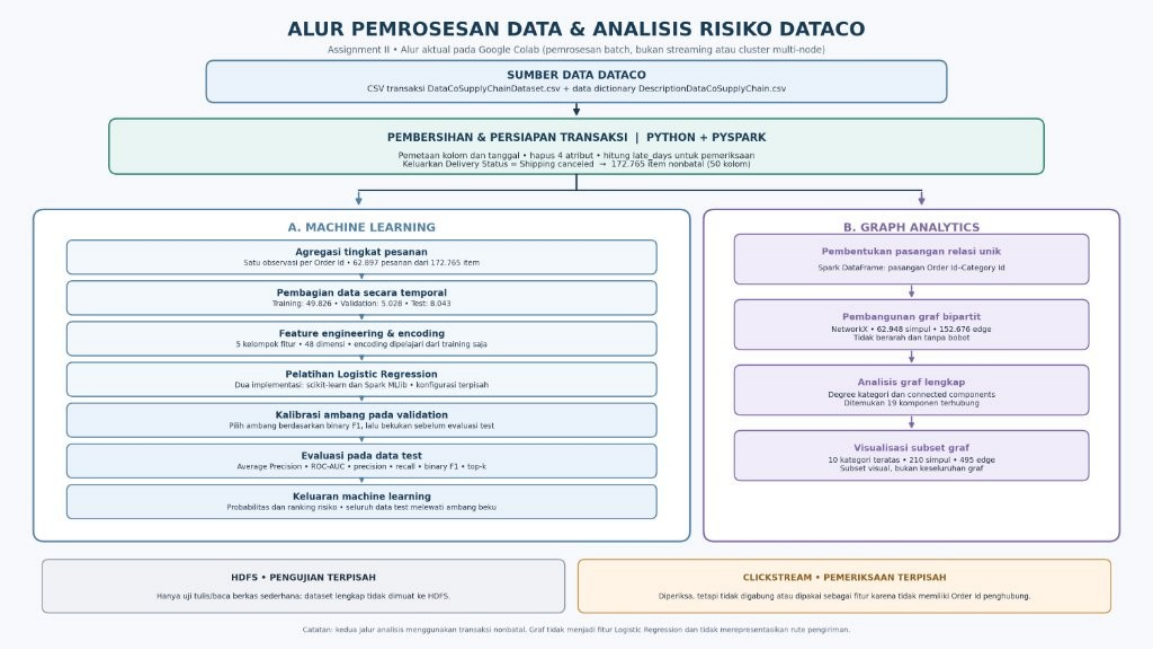
| Komponen | Kondisi aktual | Status desain |
|---|---|---|
| Apache Hadoop / HDFS | Hadoop 3.4.3; HDFS *single-node*; replication=1 | Penyimpanan lapisan data dalam rancangan |
| CSV mentah/bersih | Berkas pada *runtime* Colab tersedia dan diperiksa | *Dataset* lengkap belum dimuat ke HDFS |
| Parquet transaksi | Tidak ada konversi terverifikasi | Format target hasil pembersihan |
| JSON *clickstream* | Sumber aktual masih CSV | Format target; bukan hasil tersedia |

## 2.3 *Processing Framework*

Secara konseptual, rancangan arsitektur *Big Data* ini dipersiapkan agar kompatibel untuk pemrosesan terdistribusi (*distributed-ready*) dengan memanfaatkan abstraksi PySpark DataFrame dan *Spark MLlib Pipeline* API, sehingga memiliki potensi untuk diskalakan ke lingkungan *cluster* terdistribusi *multi-node* (seperti Apache Hadoop YARN atau Kubernetes) tanpa mengubah logika program. Namun, dalam pelaksanaan proyek ini, implementasi dieksekusi sebagai *Proof-of-Concept* (PoC) pada *runtime* tunggal Google Colab berbasis Linux x86_64 dengan Python 3.13, PySpark, dan Java OpenJDK. Apache Spark dijalankan dalam mode local[2] (dua thread pemrosesan dalam satu VM Google Colab) dengan sistem berkas lokal virtual (file:///), bukan *cluster* terdistribusi *multi-node*. Pengujian skalabilitas *multi-node* belum dilakukan dan tidak diklaim dalam laporan ini.

Spark mendukung transformasi tabular, agregasi, dan *machine learning* dalam satu ekosistem pemrosesan terpadu. *Spark MLlib* dipakai untuk melatih *Logistic Regression*, sedangkan Spark DataFrame dipakai untuk membentuk relasi unik pesanan dan kategori. Pembersihan data dan evaluasi juga memanfaatkan pustaka Python pada Google Colab. Pengukuran waktu, penggunaan memori, kapasitas pemrosesan, dan skalabilitas *multi-node* tidak dilakukan pada skala produksi. Oleh karena itu, tidak ditarik simpulan bahwa implementasi PoC pada Colab ini otomatis lebih cepat daripada pemrosesan single-thread. Dasar pemrosesan Spark mengacu pada Zaharia et al. (2012) dan dokumentasi Apache Spark (Apache Software Foundation, n.d.-b).

Gambar 2.1 merangkum alur aktual dari CSV transaksi pada *runtime* Google Colab menuju dua jalur analisis, yaitu *machine learning* dan graf pesanan–kategori. HDFS ditampilkan terpisah karena pengujiannya hanya mencakup baca dan tulis berkas sederhana. *Clickstream* tidak digunakan dalam model maupun integrasi fitur.




**Gambar 2.1 Arsitektur dan Alur Pemrosesan DataCo pada Assignment II**

Alur pengolahan mencakup pemetaan nama kolom, konversi tanggal, penghapusan empat atribut, perhitungan late_days untuk pemeriksaan, dan pemilihan pesanan nonbatal. Data lalu diagregasi menjadi satu observasi per *Order Id*, diubah menjadi fitur, dan dipakai untuk menghasilkan probabilitas serta prediksi. Penerapan daftar prioritas operasional dan pemrosesan *streaming* tidak termasuk pengujian.

### 2.4 *Repository* Kode dan Artefak Implementasi

Untuk mendukung transparansi dan keterlacakan implementasi, kode sumber dan dokumentasi pendukung Assignment II disimpan dalam repositori GitHub. *Repository* ini memuat berkas implementasi dan pengujian untuk pengolahan data, *feature engineering*, pengembangan model *Logistic Regression*, evaluasi, serta *Graph Analytics*. *Repository* tersebut dapat diakses melalui tautan berikut:

**GitHub Repository – COMP8035041 Assignment II DataCo**

*Repository* ini menjadi referensi pendukung untuk menelusuri implementasi teknis yang dibahas pada Bab 4, Bab 5, dan Bab 6.

# BAB 3. TATA KELOLA IT

## 3.1 *Security*

Konfigurasi HDFS memakai alamat *loopback* untuk NameNode dan DataNode sehingga komunikasi hanya berjalan di dalam lingkungan Colab. Pelabuhan pemrosesan tidak dibuka ke jaringan luar, dan autentikasi Kerberos tidak diaktifkan karena eksperimen berada pada lingkungan terisolasi satu pengguna. Pengamanan data bertumpu pada kontrol akses lingkungan *runtime* Google Colab dan sesi pengguna.

Pembersihan transaksi menghapus *Customer Password* dan *Customer Email* guna mengurangi eksposur kredensial dan risiko keamanan, serta membuang *Product Description* dan Product Image yang tidak terpakai.

*Dataset* DataCo versi 5 dirilis secara publik melalui Mendeley Data dengan lisensi CC BY 4.0 (Constante et al., 2019). Meskipun dapat diakses bebas untuk penelitian akademik, penggunaan *dataset* publik ini tetap menuntut penerapan etika penanganan data akademik secara cermat.

Simulasi insiden, pengujian pemulihan, dan penegakan retensi otomatis belum dilakukan sehingga kesiapan pemulihan bencana tidak diuji. Langkah yang telah dijalankan terbatas pada isolasi lokal serta pemisahan salinan data mentah dari direktori keluaran olahan.

## 3.2 *Compliance*

DataCo versi 5 dirilis dengan lisensi CC BY 4.0 (Constante et al., 2019) dan DOI 10.17632/8gx2fvg2k6.5. Lisensi ini mengizinkan penggunaan dan pengolahan data untuk keperluan analitik akademik dengan atribusi sumber yang sah. Penilaian kepatuhan pada laporan ini menunjukkan bahwa tata kelola data akademik telah terpenuhi, sedangkan kepatuhan regulasi industri seperti GDPR hanya diposisikan sebagai batas analisis komparatif.

## 3.3 Privasi Data

*General Data Protection Regulation* (GDPR, European Parliament and Council of the European Union, 2016) dipakai sebagai rujukan prinsip tata kelola data, bukan penetapan yurisdiksi atau klaim kepatuhan hukum penuh. Penghapusan empat kolom (*Customer Email*, *Customer Password*, *Product Description*, dan Product Image) dari transaksi bersih belum menjadikan seluruh *dataset* anonim, karena atribut identitas dan lokasi pelanggan lainnya masih tersimpan pada data transaksi serta alamat IP masih tercatat pada berkas *clickstream* mentah. Atribut-atribut tersebut masih memerlukan penilaian privasi tersendiri, dan *dataset* ini belum dinyatakan anonim. Lapisan analitik *clickstream* sendiri tidak dibangun dalam penelitian ini.

## 3.4 Kualitas Data

Pemeriksaan kualitas mencakup jumlah baris dan kolom, identitas unik, konversi tanggal, duplikasi baris penuh, nilai kosong, serta hubungan label dengan durasi pada cakupan nonbatal. Pemeriksaan kebaruan operasional, perubahan distribusi, rentang seluruh atribut, dan karantina otomatis tidak dilakukan. Nilai late_days negatif menunjukkan durasi aktual lebih pendek daripada jadwal, dan tidak otomatis berarti data salah.

Integritas ketiga CSV diperiksa sebelum eksperimen dengan mencocokkan kondisi berkas terhadap metadata sumber resmi, lalu menyimpan ringkasan nilai hash SHA-256. Pemeriksaan ini memastikan integritas berkas mentah sebelum diproses oleh *script* pembersihan. Pemeriksaan kualitas data disajikan pada Tabel 3.1.

**Tabel 3.1 Pemeriksaan kualitas data dan keterbatasannya**

| Pemeriksaan | Hasil pemeriksaan | Batas |
|---|---|---|
| Jumlah, tipe, nilai kosong, ID unik | Diperiksa ulang dari CSV sumber | Pemeriksaan menggunakan Python |
| Jumlah keluaran bersih | 180.519 dan 172.765 baris, masing-masing 50 kolom | Keluaran transaksi, bukan jumlah observasi model |
| Hash sumber | Metadata resmi tersedia | Ketiga hash cocok dengan sumber resmi |
| Duplikasi *clickstream* | 3.249 baris identik terverifikasi | Identik tidak membuktikan kejadian sama |
| Makna operasional label | Label konsisten dengan perbandingan durasi | Tidak menetapkan makna penyerahan fisik kepada kurir |
| Kontrol operasional | Rancangan | Belum diuji |

# BAB 4. DATA

## 4.1 Deskripsi Data

Jumlah pada Tabel 4.1 diperoleh dari pembacaan ulang berkas resmi. Ketiga berkas berbentuk CSV (comma-separated values), yaitu berkas teks sederhana yang menyusun data dalam baris dan kolom sehingga dapat dibuka program pengolah tabel. Penerbit menyebut data catatan kunjungan situs (*clickstream*) sebagai data tidak terstruktur (unstructured), padahal berkas tokenized_access_logs.csv yang kami periksa sudah berbentuk tabel dengan delapan kolom tetap. Perbedaan sebutan ini kami catat karena memengaruhi cara pengolahan, sedangkan pilihan format JSON pada rancangan tidak mengubah struktur CSV historis yang sudah ada. Tabel 4.1 dibaca per baris: kolom jumlah adalah hasil hitung ulang atas berkas, sedangkan kolom terakhir menjelaskan peran setiap berkas.

**Tabel 4.1 Sumber data**

| Berkas | Baris terverifikasi | Kolom terverifikasi | Fungsi |
|---|---|---|---|
| DataCoSupplyChainDataset.csv | 180.519 | 53 | Transaksi tingkat item |
| tokenized_access_logs.csv | 469.977 | 8 | Catatan kunjungan |
| DescriptionDataCoSupplyChain.csv | 52 | 2 | Kamus data |

Data sumber memuat 180.519 *Order Item Id*, yaitu identitas satu barang dalam pesanan, dan 65.752 *Order Id* yang unik sebagai identitas satu nomor pesanan. Rata-rata jumlah barang per pesanan dihitung pada Persamaan (4.1), sedangkan cakupan nonbatal memuat 62.897 pesanan. Satu temuan penting muncul di sini: dalam satu *Order Id*, semua barang memiliki tanggal pesanan, tanggal pengiriman, dan status keterlambatan yang sama. Karena itu keterlambatan cukup dinilai sekali untuk setiap pesanan, tanpa dihitung ulang per barang.

$$\mu = \frac{180.519}{65.752} \approx 2{,}75 \ \ item/pesanan \qquad (4.1)$$

*Clickstream* memiliki kolom Product, Category, Date, Month, Hour, Department, ip, dan url. Pemeriksaan berkas menunjukkan tidak ada nilai kosong pada kolom-kolom tersebut. *Clickstream* tidak memuat *Order Id* sehingga tautan langsung ke transaksi tidak tersedia tanpa asumsi tambahan mengenai rentang waktu pemesanan.

### 4.1.1 Pembersihan Data Transaksi

Pembersihan dimulai dari 180.519 baris tingkat barang, yaitu satu baris mewakili satu barang dalam satu pesanan. Dari jumlah tersebut, empat kolom yang tidak relevan dan sensitif dihapus: atribut privasi dan keamanan (*Customer Email* dan *Customer Password*), atribut deskripsi produk yang 100% kosong (*Product Description*), serta tautan gambar (Product Image), sehingga data transaksi menjadi 49 kolom. Selanjutnya, ditambahkan kolom audit late_days sehingga berkas memuat 50 kolom. Tahap berikutnya adalah penyaringan kohort: sebanyak 7.754 baris transaksi dengan *Delivery Status* = Shipping canceled (terdiri atas 4.062 baris *SUSPECTED_FRAUD* dan 3.692 baris CANCELED, yang mencakup 2.855 pesanan unik) dikeluarkan dari cakupan analisis karena pesanan yang dibatalkan tidak menjalani proses pengiriman fisik normal. Tahap ini menghasilkan 172.765 baris transaksi nonbatal (50 kolom) yang mencakup 62.897 pesanan unik.

**Tabel 4.2 Pemetaan nama kolom**

| Kolom sumber | Nama keluaran | Catatan |
|---|---|---|
| order date (DateOrders) | order_date | Tanggal pesanan |
| shipping date (DateOrders) | shipping_date | Tanggal pengiriman; bukan fitur prediksi |
| *Days for shipping (real)* | dispatch_real_days | Alias teknis durasi aktual; definisi mengikuti sumber |
| *Days for shipment (scheduled)* | dispatch_scheduled_days | Alias teknis durasi terjadwal; definisi mengikuti sumber |
| *Late_delivery_risk* | is_late | Label sumber tidak didefinisikan ulang |

Pada data transaksi tidak ditemukan duplikat baris penuh, sedangkan *Product Description* kosong pada seluruh 180.519 baris. Setelah *Customer Email*, *Customer Password*, *Product Description*, dan Product Image dihapus, tersisa 49 kolom sumber sebelum penambahan late_days.

Kolom late_days dihitung sebagai selisih durasi pengiriman aktual dan durasi yang dijadwalkan dalam satuan hari, sesuai Persamaan (4.2). Ilustrasi perhitungannya (bukan data penelitian): bila sebuah barang dijadwalkan tiba dalam 3 hari tetapi aktualnya 5 hari, late_days bernilai 2 hari. Penambahan kolom ini menghasilkan 50 kolom, sedangkan label is_late tetap nama lain dari *Late_delivery_risk* pada data sumber dan definisinya tidak kami ubah. Pada seluruh item nonbatal, is_late konsisten dengan kondisi durasi aktual yang melebihi durasi terjadwal. Kolom late_days dibuat hanya untuk pemeriksaan itu, karena late_days maupun durasi aktual baru diketahui setelah pengiriman terjadi sehingga keduanya tidak dipakai sebagai masukan model.

$$d_{late} = d_{actual} - d_{scheduled} \qquad (4.2)$$

Data transaksi bersih berisi 180.519 baris dan 50 kolom, dengan satu baris mewakili satu barang dalam satu pesanan. Setelah 7.754 item dengan *Delivery Status* = Shipping canceled (dibatalkan) dikeluarkan, data nonbatal memuat 172.765 baris dan 50 kolom. Sebanyak 98.977 item berlabel 1 dan 73.788 berlabel 0. Kolom status akhir hanya dipakai untuk menentukan pesanan mana yang masuk cakupan penelitian dan tidak pernah menjadi masukan model. Karena cakupan ditentukan dari informasi akhir seperti itu, hasil penelitian ini menggambarkan pesanan yang secara historis diketahui tidak dibatalkan. Hasil tersebut belum tentu berlaku sama untuk pesanan baru yang statusnya belum diketahui.

**Tabel 4.3 Tahapan pembersihan transaksi**

| Tahap | Baris terverifikasi | Kolom | Perubahan |
|---|---|---|---|
| Sumber | 180.519 | 53 | Empat kolom dihapus: *Customer Email*, *Customer Password*, *Product Description*, Product Image |
| Penghapusan empat kolom | 180.519 | 49 | Empat kolom dihapus |
| Penambahan late_days | 180.519 | 50 | 7.754 item Shipping canceled dikeluarkan (4.062 *SUSPECTED_FRAUD* + 3.692 CANCELED) |
| Cakupan nonbatal | 172.765 | 50 | 7.754 item batal dikeluarkan |

### 4.1.2 Organisasi Data

Data diorganisasi menjadi tiga lapisan agar jejak pengolahan jelas: mentah, bersih, dan siap analitik. Lapisan mentah memuat CSV asli dari sumber Mendeley Data. Lapisan bersih memuat hasil pembersihan transaksi dan verifikasi *clickstream*. Lapisan siap analitik memuat data tingkat pesanan nonbatal (62.897 observasi) serta relasi pesanan-kategori untuk analisis graf sehingga setiap tahap pemrosesan dapat diaudit secara mandiri.

**Tabel 4.4 Organisasi data**

| Lapisan | Isi | Lokasi/status |
|---|---|---|
| Mentah | Tiga CSV dan hash acuan SHA-256 | *Runtime* Google Colab; integritas diperiksa |
| Bersih | Transaksi bersih dan transaksi nonbatal | *Runtime* Google Colab; masing-masing 50 kolom |
| Siap analitik | Data pesanan, fitur, label, pembagian data, model, dan prediksi | Penyimpanan Google Colab |
| HDFS | Berkas pengujian sederhana | Uji tulis/baca; bukan penyimpanan *dataset* lengkap |

## 4.2 Model Data dan Operasi Data

Unit observasi, prediksi, dan evaluasi adalah pesanan, dengan satu baris untuk setiap *Order Id*. Artinya, 172.765 baris barang diringkas menjadi 62.897 baris pesanan. Sebagai ilustrasi, pesanan yang memuat lima barang diwakili oleh satu baris data, bukan lima baris terpisah. Sebelum penggabungan, konsistensi target, jadwal pengiriman, segmen pelanggan, *market*, dan tanggal diperiksa di dalam setiap pesanan, dan tidak ditemukan nilai yang bertentangan. Kategori produk dibentuk sebagai himpunan *Category Id* unik pada pesanan tersebut, sedangkan jumlah item dihitung dari *Order Item Id* yang unik. Penggabungan ini mencegah satu pesanan dihitung berulang sebagai beberapa observasi model, yang akan membuat jumlah data tampak lebih besar daripada jumlah kejadian sebenarnya.

Rancangan awal pada Assignment I memuat cara menggabungkan skor barang menjadi skor pesanan dengan memilih nilai probabilitas tertinggi atau nilai rata-rata barang. Pendekatan tersebut tidak digunakan pada Assignment II karena pemodelan beralih ke unit analisis tingkat pesanan sejak awal pengolahan data sehingga agregasi probabilitas pascaprediksi tidak lagi diperlukan.

$$R_o = \max_{i \in I_o} r_i \qquad (4.3)$$

Persamaan (4.4) menggambarkan rancangan lama untuk menyatukan *clickstream* dalam jendela 24 jam sebelum pemesanan. Rancangan ini tidak diimplementasikan karena *clickstream* tidak memuat identitas penghubung langsung ke pesanan sehingga penggabungan fitur perilaku web tidak dapat divalidasi secara empiris.

$$t_{order} - 24\ h \le t_{event} < t_{order} \qquad (4.4)$$

*Clickstream* tidak disertakan karena kecocokan antara catatan kunjungan dan catatan transaksi belum dapat dipastikan. *Dataset clickstream* berrentang dari September 2017 hingga Januari 2018, sedangkan transaksi mencakup periode 2015 hingga awal 2018. Tidak adanya kunci relasi unik membuat integrasi keduanya berisiko menimbulkan kebocoran informasi buatan.

**Tabel 4.5 Peran variabel**

| Kelompok | Contoh | Peran dan batas |
|---|---|---|
| Kandidat informasi sebelum pengiriman | Tepat lima kelompok: jadwal, segmen, *market*, kategori multi-hot, jumlah ID item unik | *Encoding* dipelajari dari *training*; ketersediaan tepat pada waktu operasional belum terbukti |
| Label | *Late_delivery_risk* / is_late | Target sumber; bukan prediktor dan bukan bukti penyerahan kepada kurir |
| Informasi hasil akhir | Durasi aktual; *Delivery Status*; shipping_date; late_days | Audit/cakupan; tidak menjadi fitur sebelum pengiriman |
| Atribut lain dari 50 kolom | Status pesanan, profit, identitas, dan atribut tersisa | Tidak digunakan sebagai fitur; ketersediaan operasional seluruh atribut belum diuji |

Durasi aktual, shipping_date, late_days, *Delivery Status*, dan status akhir tidak digunakan sebagai prediktor. Nilai-nilai tersebut baru terbentuk setelah barang dikirimkan sehingga pemakaiannya akan menimbulkan *data leakage*. Penelitian ini hanya memanfaatkan atribut yang sah tersedia saat pesanan dibuat.

**Tabel 4.6 Model dan operasi data**

| Sumber/keluaran | Unit/model | Operasi | Status |
|---|---|---|---|
| Transaksi | Tabular/item | Pemetaan, tanggal, empat kolom, late_days, nonbatal | Dijalankan menggunakan PySpark/Python Colab |
| *Clickstream* | Tabular/kunjungan | Struktur, *timestamp*, kosong, identik penuh | Pemeriksaan selesai; IP tetap pada sumber; deduplikasi tidak diterapkan |
| Integrasi | Agregat waktu/produk | Jendela 24 jam pra-pesanan | Desain historis Assignment I; tidak dipakai Assignment II |
| Prioritas | *Order Id* | Model langsung per pesanan | Model langsung per pesanan telah dilatih dan dievaluasi; penggunaan prioritas operasional belum diuji |
| Aliran masa depan | Kejadian berkelanjutan | Structured *Streaming* | Rancangan; tidak diuji |

Seluruh operasi pembersihan data dan pengelompokan transaksi hingga menghasilkan kohort nonbatal 172.765 item dan 62.897 pesanan unik dieksekusi secara terintegrasi pada *runtime* Google Colab langsung dari CSV mentah resmi berukuran 95,9 MB. Dengan demikian, seluruh tahapan pemrosesan data tercatat dan dapat direproduksi secara konsisten dalam lingkungan Google Colab.

Data tingkat pesanan pada bagian 4.3 menjadi dasar pengembangan model pada Bab 5 serta graf pesanan dan kategori pada Bab 6.

### 4.3 Data Tingkat Pesanan untuk Diolah Menggunakan *Machine Learning*

Data utama penelitian adalah DataCoSupplyChainDataset.csv versi 5, sedangkan berkas DescriptionDataCoSupplyChain.csv yang memuat keterangan arti setiap kolom dipakai untuk memeriksa definisi variabel. Data *clickstream* tidak digunakan dalam model maupun dalam penggabungan fitur. Cakupan penelitian mengecualikan pesanan berstatus *Delivery Status* = Shipping canceled (dibatalkan), dan kolom status itu hanya dipakai untuk memilih serta memeriksa populasi, bukan sebagai prediktor.

Pengelompokan menghasilkan 62.897 pesanan nonbatal dari 172.765 item, setelah 2.855 pesanan yang mencakup 7.754 item dikeluarkan karena memuat barang yang dibatalkan, dan tidak ditemukan pesanan dengan campuran barang batal dan nonbatal. Dari 62.897 pesanan itu, 36.048 berlabel 1 (terlambat) dan 26.849 berlabel 0 (tidak terlambat). Angka inilah yang menjadi jumlah data penelitian, dan jumlahnya mengikuti jumlah pesanan, bukan jumlah barang. Tabel 4.7 merangkum alur angka ini dari transaksi sumber hingga unit model.

Secara konseptual, model diproyeksikan untuk beroperasi pada tahap ketika pesanan telah terkonfirmasi dan siap memasuki proses pengiriman. Namun, dalam penelitian ini, penyaringan kohort dilakukan secara retrospektif menggunakan status akhir pengiriman (*Delivery Status* != Shipping canceled), bukan melalui mekanisme penyaringan pembatalan operasional waktu nyata yang diuji secara langsung. Pemilihan variabel dibatasi pada atribut yang secara logis telah ditentukan sebelum pengiriman berlangsung guna meminimalkan risiko kebocoran target, dengan tetap tunduk pada keterbatasan ketersediaan stempel waktu historis sebagaimana diuraikan pada Bab 5.6.

**Tabel 4.7 Cakupan data tingkat pesanan Assignment II**

| **Pemeriksaan** | **Hasil** |
|---|---|
| Transaksi sumber | 180.519 item; 65.752 pesanan |
| Cakupan dipertahankan | 172.765 item; 62.897 pesanan |
| Dikeluarkan | 7.754 item; 2.855 pesanan |
| Pesanan campuran batal/nonbatal | 0 |
| Target tingkat pesanan | 0 = 26.849; 1 = 36.048 |
| Unit model | Satu observasi per *Order Id*; 48 dimensi aktual |

# BAB 5. *MACHINE LEARNING*

## 5.1 Data *Preprocessing*

Prapengolahan (*preprocessing*) mengubah data mentah menjadi bentuk yang dapat diolah model. Tahap ini mencakup pemilihan kohort pesanan yang valid, transformasi fitur kategorikal dan numerik, serta penyusunan matriks masukan terstandarisasi untuk proses pelatihan model.

Integritas ketiga CSV pada eksperimen diperiksa. Data sumber memuat 180.519 item dalam 65.752 pesanan. Pada cakupan yang dipertahankan, tidak ditemukan konflik target, konflik atribut bernilai tunggal dalam pesanan, maupun kategori kosong. Temuan ini penting karena penggabungan data per pesanan mengandaikan setiap pesanan hanya memiliki satu nilai untuk target dan untuk atribut seperti jadwal, segmen, dan *market*.

Model matematika hanya dapat mengolah bilangan numerik sehingga setiap kategori produk diubah menjadi representasi biner melalui teknik pengkodean (*encoding*). Hal ini memastikan variabel kategorikal dapat diproses oleh algoritma regresi linier maupun algoritma terdistribusi Spark.

Karena satu pesanan dapat memuat beberapa kategori produk, himpunan *Category Id* memakai *multi-hot encoding* dengan dimensi sesuai kategori pada data latih. Pesanan yang memuat kategori baru yang belum pernah muncul pada data *training* dipetakan ke kolom *UNKNOWN* dengan koefisien nol sehingga stabilitas dimensi vektor tetap terjaga.

Dua fitur numerik memakai satuan asli, tanpa penskalaan atau transformasi logaritmik. Nilai numerik atau identitas wajib yang kosong atau tidak berhingga membuat pengolahan dihentikan, bukan ditebak. Seluruh aturan perubahan yang dipelajari dari data *training* yang ada diterapkan apa adanya pada *validation* dan *test* tanpa dipelajari ulang, agar model tidak melihat data uji saat belajar.

Kolom *MISSING* tidak pernah aktif di seluruh *dataset* karena tidak ada *Category Id* yang kosong pada kohort non-batal. Sebaliknya, kolom *UNKNOWN* aktif pada 99,1% pesanan di data *test*, karena 19 dari 51 kategori merupakan jenis baru yang tidak terlihat pada data *training* yang ada. Kategori yang tidak ditemukan dalam data latih tidak memiliki koefisien pengaruh yang dipelajari model karena bobot *UNKNOWN* bernilai nol. Oleh sebab itu, kemampuan model menangani kategori baru masih terbatas. Aktifnya kolom *UNKNOWN* tidak berarti seluruh prediksi kehilangan dasar: kategori yang sudah dikenal pada data *training* serta fitur lain (jadwal pengiriman, segmen pelanggan, *market*, dan jumlah item) tetap memberi kontribusi pada skor. Skor tetap terbentuk, tetapi tidak dapat menjelaskan pengaruh kategori baru secara spesifik. Pengelompokan kategori langka ke dalam kategori tunggal *OTHER* direkomendasikan untuk pengembangan berikutnya.

Rincian kuantitatifnya: 19 kategori baru tersebut adalah *Category Id* 4 dan 59 sampai 76. Pada data *test*, 7.972 dari 8.043 pesanan (99,12%) memuat paling sedikit satu kategori baru, dan 7.964 di antaranya hanya memuat kategori baru. Sebagai pembanding, pada *validation* proporsinya hanya 54 pesanan (1,07%), sedangkan pada data *training* kategori baru sama sekali tidak ditemukan. Skor rata-rata pesanan *test* yang memuat kategori baru adalah 0,5731 dengan tingkat keterlambatan 0,5773, sedangkan 71 pesanan tanpa kategori baru memiliki skor rata-rata 0,5247 dan tingkat keterlambatan 0,4789. Karena koefisien *UNKNOWN* bernilai nol, selisih skor itu berasal dari fitur lain, bukan dari kategori baru. Teknik *OTHER* sebagai pengembangan lanjutan harus dibentuk dari aturan *training*, bukan dari informasi *test*.

## 5.2 *Feature Engineering*

Model menggunakan lima kelompok fitur, yaitu *Days for shipment (scheduled)* sebagai jadwal pengiriman yang dijanjikan, *Customer Segment* untuk segmen pelanggan, *Market* untuk wilayah pemasaran, *Category Id* dengan *multi-hot encoding*, serta jumlah *Order Item Id* unik per pesanan. Jadwal pengiriman menggambarkan waktu yang dijanjikan, segmen pelanggan dan *market* menggambarkan konteks pesanan, kategori menggambarkan komposisi produk, sedangkan jumlah item menggambarkan kompleksitas pesanan. Fitur-fitur ini dibentuk dari data transaksi nonbatal yang tersedia sebelum pengiriman berlangsung.

Jumlah item menghitung baris item pesanan unik (*unique order line items*), bukan jumlah SKU (nomor identitas jenis barang)/jenis produk maupun total kuantitas barang. Dengan demikian, produk yang sama pada dua baris item berbeda tetap dihitung sebagai dua entri. Quantity, atribut keuangan, dan variabel turunannya tidak digunakan dalam model.

*Days for shipping (real)*, dispatch_real_days, shipping_date, late_days, *Delivery Status*, dan status akhir pengiriman tidak disertakan dalam fitur model. Atribut-atribut ini baru terbentuk setelah pengiriman selesai sehingga penggunaannya akan menimbulkan kebocoran target yang fatal.

**Tabel 5.1 Fitur dan representasi masukan model**

| Komponen | Representasi yang digunakan |
|---|---|
| Unit dan target | *Order Id*; *Late_delivery_risk* |
| Lima kelompok fitur | Jadwal; segmen; *market*; *Category Id* multi-hot; jumlah *Order Item Id* unik |
| Quantity dan keuangan | Tidak diaktifkan, termasuk turunannya |
| Statistik transformasi | *Vocabulary* hanya *fit* pada *training*; *MISSING* dan *UNKNOWN* |
| Matriks input final | 49.826 × 48; 5.028 × 48; 8.043 × 48; tanpa *scaling*/log |

*Encoding* menghasilkan 48 kolom masukan, yang dalam laporan ini disebut juga 48 dimensi: 2 kolom angka, 5 kolom *Customer Segment*, 7 kolom *Market*, dan 34 kolom *Category Id*. Kolom kategori itu sudah mencakup kedua penanda khusus (*MISSING* dan *UNKNOWN*), sebagaimana dijelaskan pada bagian 5.1. Matriks *training* berukuran 49.826 baris pesanan × 48 kolom, *validation* 5.028 × 48, dan *test* 8.043 × 48. Tabel 5.1 merangkum susunan masukan ini dan dibaca sebagai daftar komponen, bukan sebagai hasil evaluasi.

## 5.3 Algoritma

Penelitian menerapkan *Logistic Regression* (regresi logistik), yaitu model yang memperkirakan peluang sebuah pesanan termasuk kelas terlambat. Keluarannya berupa nilai peluang yang dapat diurutkan dari tertinggi ke terendah. Implementasinya dibuat dua kali dengan dua perangkat lunak berbeda, scikit-learn (Pedregosa et al., 2011) dan *Spark MLlib* (Meng et al., 2016), memakai parameter tetap sebagaimana tercantum pada Tabel 5.2. Keduanya memakai regularisasi L2, yaitu pengaman agar model tidak terlalu menghafal data latih, tanpa pembobotan kelas, penyeimbangan ulang data (*resampling*), atau pencarian parameter otomatis. Parameter random_state=42 dicatat dalam konfigurasi, tetapi tidak memengaruhi proses optimasi karena implementasi scikit-learn menggunakan *solver lbfgs*. Parameter tersebut hanya berlaku untuk jenis *solver sag*, *saga*, dan *liblinear*.

Reproduktibilitas eksperimen didukung melalui pencatatan versi perangkat lunak, konfigurasi model, data, dan kode pengolahan. *Baseline* adalah pembanding paling sederhana: model ini tidak melihat data pesanan sama sekali, melainkan menebak memakai proporsi kelas positif pada data *training* sebagai probabilitas konstan.

**Tabel 5.2 Parameter *Logistic Regression***

| Implementasi | Parameter |
|---|---|
| scikit-learn | C=1,0; *solver=lbfgs*; max_iter=1000; tol=0,000001; random_state=42 (tidak berpengaruh pada *lbfgs*); class_weight=None |
| *Spark MLlib* | regParam=0,1; elasticNetParam=0,0; maxIter=1000; tol=0,000001; standardization=False |
| Hubungan parameter | Tidak dinyatakan ekuivalen; dibandingkan sebagai konfigurasi primer terpisah |
| Model tambahan | *Random Forest* dan *Gradient Boosting* bersifat eksploratori dan dikeluarkan dari run final |
| Refit | Tidak melatih ulang menggunakan gabungan *training* dan *validation* |

Parameter diterapkan tanpa penyetelan hiperparameter otomatis (hyperparameter *tuning*). Implementasi scikit-learn menggunakan regularisasi C=1,0, sedangkan *Spark MLlib* menggunakan regParam=0,1. Perbedaan formulasi fungsi objektif antara kedua pustaka dicatat secara transparan sebagai karakteristik teknis pustaka.

## 5.4 Proses Pengembangan Model dengan dan tanpa *Spark MLlib*

Data dibagi menurut urutan temporal tanggal pesanan (order_date), bukan secara acak: data *training* sebelum 1 Juli 2017, data *validation* dari 1 Juli hingga 30 September 2017, dan data *test* dari 1 Oktober 2017 hingga 31 Januari 2018. Pembagian temporal ini mencerminkan skenario penerapan operasional nyata: model dilatih pada rentang waktu masa lalu dan diuji pada rentang waktu masa depan.

Setiap *Order Id* hanya berada dalam satu bagian sehingga satu pesanan tidak muncul di dua bagian sekaligus. Hal ini menjamin tidak ada kebocoran observasi antar-subset data.

**Tabel 5.3 Pembagian temporal data pesanan**

| Bagian | Label 0 | Label 1 | Jumlah |
|---|---|---|---|
| *Training* | 21.302 | 28.524 | 49.826 |
| *Validation* | 2.140 | 2.888 | 5.028 |
| *Test* | 3.407 | 4.636 | 8.043 |

Tabel 5.3 menunjukkan jumlah dan distribusi label pada setiap bagian data. Setiap observasi memiliki *Order Id* unik. Kedua implementasi menerima susunan fitur yang sama, dan probabilitas dari Spark dicocokkan kembali menggunakan *Order Id*. Model dilatih hanya pada *training*, tanpa pelatihan ulang menggunakan gabungan *training* dan *validation*.

Eksperimen final dijalankan pada Google Colab dengan Python 3.13.16 pada Linux x86_64 dan Spark 4.0.4. Spark dijalankan dengan pengaturan master local[2], artinya komputasi memakai dua thread pemrosesan lokal dalam satu proses di satu mesin (bukan dua prosesor fisik), bukan banyak komputer yang saling terhubung (*multi-node*). Penyimpanan memakai penyimpanan biasa mesin virtual Colab (*filesystem* lokal), bukan HDFS. Konfigurasi ini mendokumentasikan lingkungan eksekusi, bukan bukti skalabilitas *multi-node*.

Seluruh angka yang dilaporkan pada Bab 5 diambil dari eksekusi final pada Google Colab ini. Pemeriksaan keterlacakan dan reproduktibilitas dilakukan dengan memverifikasi berkas metrik keluaran JSON dan larik probabilitas yang dihasilkan langsung dari *pipeline* Colab.

Eksekusi final menghasilkan model, probabilitas, prediksi, ambang *validation*, metrik, dan statistik graf. Data utama berukuran 95.910.149 byte (sekitar 91,5 MiB). Seluruh pelatihan, eksekusi Spark, dan komputasi graf dilakukan pada *runtime* Google Colab. Pengukuran waktu eksekusi dan uji skalabilitas *multi-node* tidak dilakukan pada eksperimen ini. Berkas pendukung eksperimen tersimpan bersama laporan ini, meliputi skrip pengolahan, keluaran JSON metrik, berkas prediksi per model, dan daftar checksum berkas pada SHA256SUMS.txt di direktori analisis-lanjutan/.

Tabel 5.4 menegaskan bahwa konfigurasi scikit-learn C=1,0 dan Spark regParam=0,1 adalah dua konfigurasi final yang terpisah. Hasil keduanya dibandingkan pada pembagian data dan representasi fitur yang sama, tanpa klaim kesetaraan parameter.

**Tabel 5.4 Konfigurasi final dua implementasi *Logistic Regression***

| Implementasi | Konfigurasi final | Status perbandingan | Batas klaim |
|---|---|---|---|
| scikit-learn | C=1,0 | Primer | Tidak ekuivalen dengan regParam Spark |
| *Spark MLlib* | regParam=0,1 | Primer | Hasil dibandingkan secara deskriptif |

Perbedaan kecil pada AP, ukuran kualitas urutan skor risiko, dan ROC-AUC dibaca sebagai hasil deskriptif dari dua implementasi dengan regularisasi berbeda. Tidak ada uji inferensial untuk menyatakan salah satu implementasi unggul.

## 5.5 Evaluasi Model

Ukuran ini penting bagi tim operasional karena menentukan pesanan mana yang ditangani lebih dulu. Bagian ini menjelaskan ukuran yang dipakai untuk menilai model. Empat hitungan berikut dihitung dari keputusan model pada satu ambang. *True positive* (TP) berarti pesanan benar-benar terlambat dan ditandai berisiko. *False positive* (FP) berarti pesanan sebenarnya tidak terlambat tetapi ditandai berisiko. *False negative* (FN) berarti pesanan terlambat tetapi tidak ditandai. *True negative* (TN) berarti pesanan tidak terlambat dan tidak ditandai.

Dari hitungan itu, *precision* adalah TP/(TP+FP), yaitu dari yang ditandai berisiko berapa bagian yang benar. *Recall* adalah TP/(TP+FN), yaitu dari yang benar-benar terlambat berapa bagian yang berhasil ditandai. *Binary F1* adalah 2TP/(2TP+FP+FN) untuk kelas 1, bukan *weighted F1*, yaitu gabungan *precision* dan *recall*. *Precision* menangkap keandalan daftar peringatan, *recall* menangkap berapa banyak keterlambatan nyata yang berhasil dikenali, dan *binary F1* merangkum keseimbangan keduanya. *Confusion matrix* adalah tabel silang antara prediksi dan kenyataan. Keempat kotaknya berisi TN, FP, FN, dan TP, lalu dibaca berurutan *true negative* (TN), *false positive* (FP), *false negative* (FN), *true positive* (TP). Kotak-kotak itu memperlihatkan titik lemah model, misalnya jumlah keterlambatan yang terlewat. Untuk *precision*, *recall*, dan F1, perhitungan memakai parameter zero_division=0, yaitu kesepakatan bahwa hasil yang tidak terdefinisi akibat pembagian dengan nol ditetapkan menjadi nol.

Dua ukuran lain menilai kualitas urutan skor tanpa bergantung pada satu ambang. *Average Precision* (AP), dihitung dengan average_precision_score, merupakan metrik utama pemeringkatan kelas positif terlambat (label 1). AP merangkum *precision* dengan pembobotan kenaikan *recall*. AP menunjukkan kemampuan model menempatkan pesanan yang benar-benar terlambat pada urutan teratas daftar risiko. Semakin tinggi nilai AP, semakin efektif tim operasional menemukan pesanan bermasalah pada kelompok prioritas teratas. AP bukan ROC-AUC, dan bukan pula luas trapesium di bawah kurva *precision-recall*. *Receiver Operating Characteristic Area Under the Curve* (ROC-AUC) mengukur pemeringkatan positif terhadap negatif, yaitu kemampuan model membedakan pesanan terlambat dan tidak terlambat pada seluruh nilai ambang.

Ambang klasifikasi adalah angka batas peluang: pesanan berpeluang sama atau lebih tinggi daripada batas tersebut dikelompokkan sebagai kelas 1 (berisiko terlambat), sedangkan pesanan di bawah batas dikelompokkan sebagai kelas 0 (tidak berisiko terlambat).

**Tabel 5.5 Hasil evaluasi pada data *test***

| Ukuran | ***Baseline*** | LR scikit-learn | LR *Spark MLlib* |
|---|---|---|---|
| *Average precision* | 0,576402 | 0,736424 | 0,736851 |
| ROC-AUC | 0,500000 | 0,719782 | 0,720601 |
| *Precision* kelas 1 | 0,576402 | 0,576402 | 0,576402 |

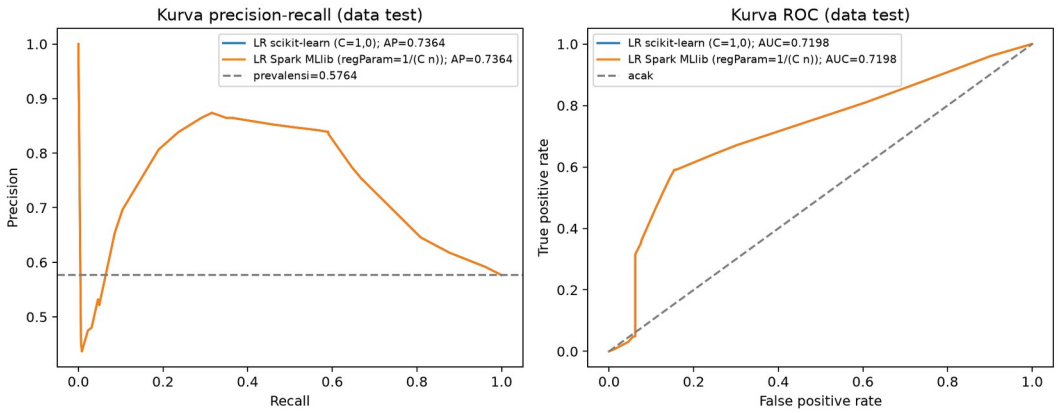
| Ukuran | *Baseline* | LR scikit-learn | LR *Spark MLlib* |
|---|---|---|---|
| *Recall* kelas 1 | 1,000000 | 1,000000 | 1,000000 |
| *Binary F1* | 0,731288 | 0,731288 | 0,731288 |
| *Confusion matrix (TN, FP, FN, TP)* | TN=0, FP=3.407 FN=0, TP=4.636 | TN=0, FP=3.407 FN=0, TP=4.636 | TN=0, FP=3.407 FN=0, TP=4.636 |
| Threshold *validation* | 0,572472 | 0,307694 | 0,446602 |

Tabel 5.5 dan 5.6 dibaca dengan setiap kolom sebagai satu model dan setiap baris sebagai satu ukuran evaluasi. Dua baris teratas, *Average Precision* (AP) dan kurva ROC (*receiver operating characteristic*), menilai kualitas pemeringkatan skor tanpa bergantung pada ambang tertentu. Baris lainnya mengikuti ambang validasi yang telah dibekukan. Angka *precision*, *recall*, dan *binary F1* yang identik pada Tabel 5.5 terjadi karena seluruh pesanan data uji terprediksi positif pada ambang tersebut.

**Tabel 5.6 Hasil evaluasi pada data *validation***

| Ukuran | *Baseline* | LR scikit-learn | LR *Spark MLlib* |
|---|---|---|---|
| *Average precision* | 0,574383 | 0,744407 | 0,744407 |
| ROC-AUC | 0,500000 | 0,719207 | 0,719909 |
| *Precision* kelas 1 | 0,574383 | 0,574383 | 0,574612 |
| *Recall* kelas 1 | 1,000000 | 1,000000 | 1,000000 |
| *Binary F1* | 0,729661 | 0,729661 | 0,729846 |
| *Confusion matrix (TN, FP, FN, TP)* | TN=0, FP=2.140 FN=0, TP=2.888 | TN=0, FP=2.140 FN=0, TP=2.888 | TN=2, FP=2.138 FN=0, TP=2.888 |
| *Threshold validation* | 0,572472 | 0,307694 | 0,446602 |

Gambar 5.1 menyajikan kurva *Precision-Recall* dan ROC pada data *test* untuk *Logistic Regression* (LR) scikit-learn dan *Spark MLlib*. Kurva *Precision-Recall* menunjukkan bahwa kedua implementasi mempertahankan presisi tinggi di atas garis dasar *baseline* (0,576402) pada fraksi pemanggilan teratas. Temuan ini mengonfirmasi efektivitas model dalam memprioritaskan pesanan berisiko tinggi keterlambatan pengiriman.




**Gambar 5.1 Kurva *Precision-Recall* dan Kurva ROC pada data *test*: scikit-learn vs *Spark MLlib***

## 5.6 Analisis

Pada data *test*, AP *baseline* adalah 0,576402. *Logistic Regression* scikit-learn menghasilkan AP 0,736424, sedangkan *Spark MLlib* menghasilkan AP 0,736851. Peningkatan AP sebesar +0,16 di atas *baseline* menunjukkan kemampuan pemeringkatan risiko yang konsisten antara kedua pustaka pada data uji.

Pada ambang *validation*, seluruh 8.043 pesanan *test* diprediksi sebagai kelas 1 (berisiko terlambat) oleh *baseline* dan kedua implementasi. Hasil *confusion matrix* pada data *test* mencatat TN=0 (tidak ada pesanan tepat waktu yang dikenali benar), FP=3.407 (pesanan tepat waktu ikut ditandai berisiko), FN=0 (tidak ada keterlambatan yang terlewat), dan TP=4.636. *Precision* mencapai 0,576402, *recall* 1,000000, dan *binary F1* 0,731288. Secara praktis, seluruh keterlambatan tertangkap, tetapi 3.407 peringatan palsu menambah beban verifikasi bagi tim operasional.

Pada *validation*, Spark masih mengklasifikasikan dua observasi negatif dengan benar, tetapi aturan ambangnya tetap hampir seluruhnya positif, jadi model belum bisa menolak pesanan yang aman secara luas. Karena itu, model menunjukkan kapasitas untuk diuji sebagai alat pemeringkatan prioritas risiko, bukan untuk memutuskan ya atau tidak secara otomatis.

Hasil pengujian *top-k* menunjukkan presisi yang kuat pada kelompok prioritas. Angka ini dihitung dari skor peluang *Logistic Regression* scikit-learn (konfigurasi pada Tabel 5.2). Spark MLlib menghasilkan pola yang hampir sama, yaitu 1.088 pesanan dengan *precision* 0,8061, 1.671 pesanan dengan *precision* 0,8737, dan 2.488 pesanan dengan *precision* 0,8533 pada batas persentil yang sama.

Pada batas persentil 10% dengan seluruh *ties* disertakan, terpilih 1.091 pesanan atau 13,56% dari 8.043 pesanan *test* (3,56 poin persentase di atas target 10%), dengan TP=880 dan *precision* 0,8066. Pada batas persentil 20% terpilih 1.674 pesanan atau 20,81% (TP=1.463, *precision* 0,8740), dan pada batas persentil 30% terpilih 2.496 pesanan atau 31,03% (TP=2.128, *precision* 0,8526). Ketiganya menghasilkan *lift*, yaitu *precision* dibagi prevalensi kelas pada data *test* (0,5764): 1,40 pada batas 10%, 1,52 pada batas 20%, dan 1,48 pada batas 30%.

Selisih cakupan aktual terjadi karena *ties*, yaitu beberapa pesanan memiliki skor yang sama pada batas persentil. Karena semua pesanan dengan skor batas itu ikut disertakan, jumlah terpilih tidak terpotong tepat pada 10%, 20%, atau 30% dari 8.043 pesanan. Sebagai gambaran praktis, *precision* 0,8740 berarti sekitar 87 dari setiap 100 pesanan yang masuk kelompok 20% teratas benar-benar terlambat pada data *test*. Nilai *baseline* 0,5764 identik dengan proporsi kelas positif (*late delivery*) pada data *test*, yaitu 4.636 dari 8.043 pesanan.

**Tabel 5.7 Ringkasan pengujian *top-k* pada data *test***

| *Cutoff* | Implementasi | Pesanan terpilih | Cakupan aktual | *True positive* | *Precision* | *Lift* |
|---|---|---|---|---|---|---|
| 10% | scikit-learn | 1.091 | 13,56% | 880 | 0,8066 | 1,40 |
| 10% | *Spark MLlib* | 1.088 | 13,53% | 877 | 0,8061 | 1,40 |
| 20% | scikit-learn | 1.674 | 20,81% | 1.463 | 0,8740 | 1,52 |
| 20% | *Spark MLlib* | 1.671 | 20,78% | 1.460 | 0,8737 | 1,52 |
| 30% | scikit-learn | 2.496 | 31,03% | 2.128 | 0,8526 | 1,48 |
| 30% | *Spark MLlib* | 2.488 | 30,93% | 2.123 | 0,8533 | 1,48 |

Kedua implementasi meningkatkan kemampuan pemeringkatan risiko dibandingkan *baseline*. Pada data *test*, *Average Precision* mencapai sekitar 0,736, jauh melampaui prior kelas mayoritas (0,576). Capaian ini mengonfirmasi keandalan model sebagai instrumen pemeringkat prioritas operasional.

Peningkatan kemampuan pemeringkatan ini belum berarti keputusan klasifikasi biner pada satu ambang tunggal telah efektif. Seluruh pesanan *test* terklasifikasi sebagai berisiko pada ambang validasi sehingga model tidak dapat difungsikan sebagai pemutus otomatis tanpa intervensi manusia.

Penyebab seluruh 8.043 pesanan *test* berada di atas ambang perlu dijelaskan secara transparan. Ambang yang dioptimalkan pada *validation* menghasilkan angka ambang yang relatif rendah (0,307694 untuk scikit-learn dan 0,446602 untuk *Spark MLlib*). Akibatnya, seluruh distribusi skor pada data *test* berada di atas batas ambang tersebut.

Mekanisme ambang *validation* yang memaksimalkan *binary F1* pada data *validation* menghasilkan angka ambang yang berada di bawah nilai probabilitas prediksi terendah pada data *test*. Dengan

demikian, hasil semua-positif pada data *test* merupakan konsekuensi mekanis dari pembekuan ambang *validation*, bukan kesalahan kalkulasi. Hasil ini menegaskan bahwa model belum efektif sebagai pemutus biner otomatis. Namun, model terbukti andal dalam menyusun urutan prioritas risiko bagi tim operasional logistik.

**Tabel 5.8 Distribusi skor peluang: *test dan validation pada eksekusi Google Colab***

| Lingkungan | Implementasi dan Data | n | Minimum | Median | Maksimum | Skor ≥ ambang |
|---|---|---|---|---|---|---|
| Colab | scikit-learn, *test* | 8.043 | 0,353144 | 0,420483 | 0,914649 | 8.043 (100,00%) |
| Colab | *Spark MLlib, test* | 8.043 | 0,447808 | 0,453735 | 0,865002 | 8.043 (100,00%) |
| Colab | scikit-learn, *validation* | 5.028 | 0,307694 | 0,426947 | 0,935811 | 5.028 (100,00%) |
| Colab | *Spark MLlib, validation* | 5.028 | 0,446311 | 0,453239 | 0,865781 | 5.026 (99,96%) |

Tabel 5.8 menyajikan ringkasan distribusi statistik skor peluang hasil prediksi model pada data *test* dan *validation* di lingkungan Google Colab. Nilai minimum dan maksimum menunjukkan rentang skor yang dipelajari masing-masing implementasi, dengan median berada pada kisaran 0,42–0,45. Skor probabilitas ini berfungsi sebagai skor pemeringkat risiko relatif, bukan estimasi probabilitas absolut terkalibrasi.

Nilai minimum dan maksimum menunjukkan skor prediksi berada pada rentang yang berbeda untuk setiap implementasi, sedangkan median berada pada kisaran 0,42–0,45. Median itu berada di bawah proporsi kelas positif pada data *test*, yaitu sekitar 0,5764. Perbandingan median dan prevalensi kelas belum cukup untuk menentukan apakah probabilitas model sudah terkalibrasi. Skor probabilitas dalam penelitian ini karena itu dipakai untuk mengevaluasi kemampuan pemeringkatan risiko, bukan sebagai estimasi probabilitas absolut yang telah terbukti akurat. Bagi tim operasional, skor boleh diurutkan untuk prioritas, tetapi tidak boleh dikutip sebagai angka peluang keterlambatan sebuah pesanan. Pengujian kalibrasi secara khusus, misalnya menggunakan *calibration curve* dan *Brier score*, dapat dilakukan pada penelitian lanjutan.

Seluruh nilai *Average Precision*, ROC-AUC, *precision*, *recall*, *binary F1*, dan *confusion matrix* pada Tabel 5.5 dan 5.6 cocok secara presisi dengan berkas artefak JSON eksekusi final di Google Colab (evidence-colab-final-2026-10-22.json). Hal ini memastikan bahwa seluruh hasil evaluasi model yang disajikan dapat ditelusuri kembali secara transparan ke artefak komputasi aslinya.

Model final dibatasi pada *Logistic Regression*. *Random Forest* (Breiman, 2001) dan *Gradient Boosting* (Friedman, 2001), uji tanpa sebagian fitur (*ablation*), serta variasi representasi jadwal pernah dieksplorasi. Ketiganya tidak menjadi bagian dari uji utama (run konfirmatori) dan tidak dipakai untuk memilih model utama.

**Tabel 5.9 Hasil pengujian ablasi kelompok fitur pada data *test***

| Kelompok Fitur | Jumlah Fitur | AP *Test* | ROC-AUC *Test* | Interpretasi Kontribusi |
|---|---|---|---|---|
| Semua fitur (model penuh) | 48 | 0,7364 | 0,7198 | Model terintegrasi (jadwal, pesanan, kategori, pasar, segmen) |
| Jadwal saja | 1 | 0,7249 | 0,7159 | Fitur *Days for shipment (scheduled)* menyumbang daya prediktif utama |
| Jadwal + jumlah item | 2 | 0,7255 | 0,7164 | Penambahan jumlah item memberikan sedikit peningkatan |

| | | | | |
|---|---|---|---|---|
| | | | | AP (+0,0006) |
| Tanpa jadwal | 47 | 0,5836 | 0,5073 | Ketiadaan fitur jadwal menyebabkan penurunan tajam AP (-0,1528) |
| Segmen + *market* saja | 12 | 0,5818 | 0,5078 | Sinyal kontekstual pasar dan segmen pelanggan |
| Kategori saja | 34 | 0,5766 | 0,5000 | Sinyal kategori marginal, mendekati garis dasar *baseline* |
| *Baseline* (prior) | 0 | 0,5764 | 0,5000 | Ambang frekuensi dasar proporsi kelas mayoritas |

Hasil studi ablasi pada Tabel 5.9 menunjukkan bahwa daya pembeda model *Logistic Regression* didominasi oleh fitur jadwal pengiriman yang dijanjikan (*Days for shipment (scheduled)*), yang secara mandiri menghasilkan *Average Precision* 0,7249 (ROC-AUC 0,7159). Fitur ini merupakan durasi terjadwal yang sah diketahui sebelum pengiriman berlangsung, bukan durasi aktual (*Days for shipping (real)*) yang baru terbentuk pascapengiriman, sehingga terbebas dari potensi kebocoran target (target leakage). Sebaliknya, penghapusan fitur jadwal (tanpa jadwal) mengakibatkan AP merosot menjadi 0,5836, mendekati *baseline* tanpa fitur (0,5764). Temuan empiris ini secara tuntas menjelaskan mengapa performa pemeringkatan model tidak runtuh meskipun 7.972 dari 8.043 pesanan *test* (99,12%) memuat kategori produk baru yang tidak dikenal dalam data latih. Fitur kategori produk sendiri hanya menyumbang AP marjinal sebesar 0,5766 (ROC-AUC 0,5000) sehingga stabilitas pemeringkatan risiko pada data *test* praktis ditopang kokoh oleh variabel jadwal pengiriman.

Ketersediaan fitur pada waktu prediksi tidak dapat direkonstruksi dari potret data historis yang tersedia. Empat kelompok fitur, yaitu jadwal, segmen pelanggan, *market*, dan kategori, tidak memiliki catatan waktu perubahan, jadi nilai fitur yang berlaku saat prediksi dibuat tidak diketahui. Jumlah *Order Item Id* unik juga mengandaikan seluruh item pesanan sudah diketahui pada saat itu. Hasilnya karena itu hanya mendukung penilaian atas data masa lalu, bukan klaim kesiapan sistem untuk dipakai dalam produksi.

Kesimpulan ini juga terbatas karena tiga hal. Pertama, data dibatasi pada pesanan yang secara historis tidak dibatalkan. Kedua, urutan waktu memakai order_date sebagai perkiraan. Ketiga, kematangan label tidak diketahui, artinya tim peneliti tidak tahu kapan status setiap pesanan benar-benar final. *Dataset* tidak memuat kolom waktu yang mencatat saat status akhir ditetapkan, sedangkan pengiriman sendiri berlangsung setelah tanggal pesanan. Metrik hanya berlaku kondisional untuk pesanan yang secara retrospektif diketahui tidak dibatalkan. Pengujian untuk penggunaan nyata memerlukan definisi kapan prediksi dilakukan, rekonstruksi nilai fitur pada saat itu, pencatatan waktu ketersediaan label, dan pengujian ke depan yang terpisah.

# BAB 6. *GRAPH ANALYTICS*

## 6.1 Pengembangan Graph

Dalam data transaksi, satu pesanan dapat memuat beberapa kategori produk sekaligus. Hubungan konkret ini dibaca sebagai graf: setiap pesanan dan setiap kategori produk menjadi simpul (*node*), sedangkan garis penghubung (*edge*) menandai kategori yang tercatat dalam pesanan tersebut. Graf ini disebut bipartit karena hanya memiliki dua jenis simpul, yaitu pesanan dan kategori. Garis penghubung hanya bertemu pada simpul pesanan dan simpul kategori, tidak pernah menghubungkan sesama pesanan atau sesama kategori. Setiap simpul diberi awalan order: dan category: agar identitas keduanya tidak tertukar. Graf tidak berarah dan tidak berbobot. Garis yang menghubungkan suatu simpul dengan dirinya sendiri (*self-loop*) tidak dipakai. Spark DataFrame membentuk garis penghubung unik, sedangkan NetworkX (Hagberg et al., 2008) menghitung metrik pada satu mesin. Graf tidak dipakai sebagai fitur model dan tidak menggambarkan rute pengiriman.

Graf lengkap memuat 62.948 simpul, terdiri atas 62.897 pesanan dan 51 kategori, dengan 152.676 garis penghubung. Struktur bipartit ini merefleksikan relasi belanja pelanggan secara menyeluruh pada *dataset* transaksi nonbatal.

**Tabel 6.1 Definisi graf pesanan dan kategori**

| Komponen | Definisi |
|---|---|
| *Node* pesanan | Satu *Order Id* yang dipertahankan pada kohort nonbatal |
| *Node* kategori | Satu *Category Id* yang muncul pada pesanan tersebut |
| *Edge* bipartit | Pasangan unik pesanan–kategori tanpa bobot dan tanpa arah |
| Metrik final | *Degree* kategori dan *connected components* |

Jumlah hubungan (*degree*) sebuah kategori adalah banyaknya pesanan unik yang terhubung ke kategori tersebut, bukan jumlah item atau unit barang. Visualisasi memakai sepuluh kategori dengan jumlah hubungan terbesar. Jika jumlah hubungannya sama, *Category Id* diurutkan secara numerik menaik. Gabungan pesanan tetangga diurutkan berdasarkan *Order Id* numerik menaik, kemudian diambil 200 pesanan pertama. Posisi simpul ditentukan dengan metode penataan pegas (*spring layout*) agar pola hubungan antar-simpul tampak jelas, bukan berdasarkan koordinat geografis fisik. Pengaturan ini memakai *seed*=42 dan *iterations*=50. Seluruh metrik graf lengkap dihitung sebelum subset dipilih.

Visualisasi subset membantu memperlihatkan keterhubungan pesanan dengan kategori produk, tetapi tidak mencakup seluruh graf. Pemilihan berdasarkan jumlah hubungan dan urutan *Order Id* bukan merupakan sampel acak. Nilai tambah analitik graf bipartit terletak pada pemetaan struktur topologi keterhubungan: graf mengungkap keberadaan 19 komponen terhubung, yang terdiri atas 1 komponen raksasa (54.966 simpul) serta 18 komponen lain masing-masing berisi satu kategori beserta pesanan yang hanya terhubung lewat kategori tersebut. Ekstraksi pasangan relasi unik pesanan-kategori dijalankan melalui operasi Spark DataFrame pada *runtime* Google Colab, sedangkan pembentukan graf dan perhitungan metrik (*degree*, komponen terhubung) dieksekusi menggunakan pustaka NetworkX pada memori mesin tunggal, bukan melalui mesin pemrosesan graf terdistribusi seperti GraphX.

**Tabel 6.2 Jumlah hubungan sepuluh kategori pada graf lengkap**

| *Category Id* | *Degree* (pesanan unik) |
|---|---|
| 17 (Cleats) | 19.534 |
| 18 (Men's Footwear) | 17.954 |
| 24 (Women's Apparel) | 17.088 |
| 46 (Indoor/Outdoor Games) | 15.912 |
| 45 (Fishing) | 14.527 |
| 48 (Water Sports) | 13.174 |
| 43 (Camping & Hiking) | 11.787 |
| 9 (Cardio Equipment) | 10.857 |
| 29 (Shop By Sport) | 9.700 |
| 37 (Electronics) | 1.920 |

**Tabel 6.3 Komponen terhubung pada graf lengkap**

| Komponen | *Node* | Pesanan | Kategori | *Edge* |
|---|---|---|---|---|
| 1 | 54.966 | 54.933 | 33 | 144.712 |
| 2 | 805 | 804 | 1 | 804 |
| 3 | 632 | 631 | 1 | 631 |
| 4 | 623 | 622 | 1 | 622 |
| 5 | 563 | 562 | 1 | 562 |
| 6 | 508 | 507 | 1 | 507 |
| 7 | 473 | 472 | 1 | 472 |
| 8 | 467 | 466 | 1 | 466 |
| 9 | 460 | 459 | 1 | 459 |
| 10 | 459 | 458 | 1 | 458 |
| 11 | 426 | 425 | 1 | 425 |
| 12 | 418 | 417 | 1 | 417 |
| 13 | 410 | 409 | 1 | 409 |
| 14 | 392 | 391 | 1 | 391 |
| 15 | 347 | 346 | 1 | 346 |
| 16 | 337 | 336 | 1 | 336 |
| 17 | 264 | 263 | 1 | 263 |
| 18 | 199 | 198 | 1 | 198 |
| 19 | 199 | 198 | 1 | 198 |

Analisis final tetap mengikuti rancangan G1 yang dirumuskan pada Assignment I, yaitu graf bipartit pesanan-kategori, jumlah hubungan tiap kategori, dan kelompok simpul terhubung. Proyeksi kategori, PageRank (Page et al., 1999), *eigenvector centrality*, *betweenness*, dan *community detection* (Blondel et al., 2008) tidak dipakai dalam analisis final.

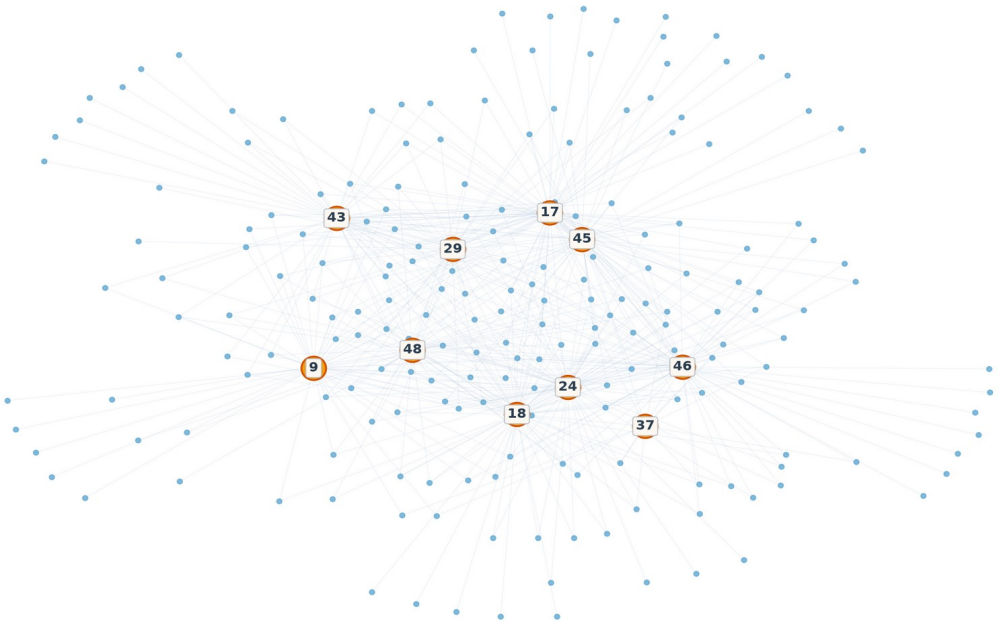
### 6.2 Analisis dan Insight yang Diperoleh

Graf lengkap terbagi menjadi 19 komponen terhubung. Kelompok terbesar memuat 54.966 simpul, terdiri atas 54.933 pesanan dan 33 kategori, dengan 144.712 garis penghubung. Delapan belas komponen lain masing-masing berisi satu kategori beserta pesanan yang hanya terhubung lewat kategori tersebut. Struktur ini menggambarkan kategori yang muncul bersama dalam pesanan, bukan wilayah, rute, atau hubungan sebab akibat.

*Category Id* 17 memiliki jumlah hubungan terbesar, yaitu 19.534 pesanan unik. Posisi berikutnya ditempati *Category Id* 18 dengan 17.954 pesanan dan *Category Id* 24 dengan 17.088 pesanan. Jumlah hubungan mengukur jangkauan kategori terhadap pesanan, bukan volume unit barang atau pengaruh kausal terhadap keterlambatan. Interpretasi ini dibatasi pada struktur graf G1 yang dihitung dari kohort retrospektif nonbatal.

Sepuluh kategori dengan jumlah hubungan tertinggi dipilih dari 51 kategori yang muncul pada graf. Pemilihan itu hanya untuk visualisasi dan tidak mengubah statistik graf lengkap. Uji kategori saja hanya memakai *Category Id* sebagai masukan, tanpa satu pun statistik graf. Implementasinya memakai scikit-learn dengan C=1,0, AP 0,576577, dan ROC-AUC 0,5000. Hasil itu konsisten dengan temuan bahwa struktur kategori tidak memisahkan keterlambatan, meskipun tingkat keterlambatan per kategori berada pada rentang 0,4963–0,6885. Variasi tersebut belum cukup untuk menghasilkan pemeringkatan pesanan yang baik. Metrik graf belum layak dijadikan fitur tanpa pengujian tambahan. Uji ini bukan evaluasi model berbasis struktur graf karena jumlah hubungan, kelompok simpul terhubung, dan representasi graf lain tidak dimasukkan sebagai fitur.

Visualisasi subset membantu memperlihatkan hubungan pesanan dengan kategori, tetapi tidak mencakup seluruh graf. Pemilihan berdasarkan jumlah hubungan dan urutan *Order Id* bukan sampel acak. Jumlah komponen dan jumlah hubungan tiap kategori ditafsirkan dari pengukuran graf lengkap, bukan dari posisi atau kepadatan simpul visual subset.




**Gambar 6.1 Subset graf pesanan dan kategori: 210 simpul dan 495 garis penghubung (Simpul biru: pesanan, simpul oranye: kategori produk, garis: keberadaan kategori dalam pesanan)**

Relasi yang dianalisis menunjukkan kategori yang tercatat bersama dalam pesanan. Graf tidak memodelkan gudang, kurir, urutan perjalanan, atau waktu tempuh. Hubungan struktur graf dengan label diuji secara deskriptif melalui tingkat keterlambatan per kategori pada Tabel 6.4. Analisis ini bukan pengujian kausal dan bukan penggunaan graf sebagai fitur model. Penyebut Tabel 6.4 berjumlah 152.676 pasangan pesanan-kategori, yaitu jumlah garis penghubung pesanan-kategori, bukan 62.897 pesanan unik. Tingkat keterlambatan per kategori berkisar 0,4963 (*Category Id* 30) sampai 0,6885 (*Category Id* 34), sedangkan tingkat keterlambatan seluruh 62.897 pesanan unik sebesar 0,5731.

Karena satu pesanan dapat memuat lebih dari satu kategori produk, pesanan yang sama dapat muncul pada beberapa baris kategori. Karena itu, penjumlahan kolom pesanan terlambat pada seluruh kategori sebesar 87.551 tidak menunjukkan jumlah pesanan terlambat yang unik.

Uji eksplorasi memakai *Logistic Regression* scikit-learn (C=1,0) pada satu set fitur kategori saja dan *multi-hot encoding* menghasilkan AP *test* 0,576577 (ROC-AUC 0,499989). Hasil ini menunjukkan bahwa informasi kategori saja tidak memberikan daya pembeda yang cukup untuk memprediksi keterlambatan pengiriman tanpa dukungan fitur jadwal.

**Tabel 6.4 Tingkat keterlambatan per kategori pada graf lengkap**

| Category Id | Pesanan unik | Pesanan terlambat | Tingkat keterlambatan |
|---|---|---|---|
| 2 (Soccer) | 134 | 74 | 0,5522 |
| 3 (Baseball & Softball) | 603 | 347 | 0,5755 |
| 4 (Basketball) | 62 | 35 | 0,5645 |
| 5 (Lacrosse) | 327 | 205 | 0,6269 |
| 6 (Tennis & Racquet) | 313 | 182 | 0,5815 |
| 7 (Hockey) | 583 | 327 | 0,5609 |
| 9 (Cardio Equipment) | 10857 | 6206 | 0,5716 |
| 10 (Strength *Training*) | 109 | 64 | 0,5872 |
| 11 (Fitness Accessories) | 294 | 174 | 0,5918 |
| 12 (Boxing & MMA) | 400 | 237 | 0,5925 |
| 13 (Electronics) | 1057 | 617 | 0,5837 |
| 16 (As Seen on TV!) | 66 | 39 | 0,5909 |
| 17 (Cleats) | 19534 | 11228 | 0,5748 |
| 18 (Men's Footwear) | 17954 | 10247 | 0,5707 |
| 24 (Women's Apparel) | 17088 | 9723 | 0,5690 |
| 26 (Girls' Apparel) | 1128 | 659 | 0,5842 |
| 29 (Shop By Sport) | 9700 | 5582 | 0,5755 |
| 30 (Men's Golf Clubs) | 272 | 135 | 0,4963 |
| 31 (Women's Golf Clubs) | 170 | 97 | 0,5706 |
| 32 (Golf Apparel) | 428 | 229 | 0,5350 |
| 33 (Golf Shoes) | 501 | 291 | 0,5808 |
| 34 (Golf Bags & Carts) | 61 | 42 | 0,6885 |
| 35 (Golf Gloves) | 1023 | 598 | 0,5846 |
| 36 (Golf Balls) | 1373 | 787 | 0,5732 |
| 37 (Electronics) | 1920 | 1122 | 0,5844 |
| 38 (Kids' Golf Clubs) | 348 | 199 | 0,5718 |
| 40 (Accessories) | 1670 | 999 | 0,5982 |
| 41 (Trade-In) | 917 | 534 | 0,5823 |
| 43 (Camping & Hiking) | 11787 | 6719 | 0,5700 |
| 44 (Hunting & Shooting) | 420 | 245 | 0,5833 |
| 45 (Fishing) | 14527 | 8330 | 0,5734 |
| 46 (Indoor/Outdoor Games) | 15912 | 9113 | 0,5727 |
| 48 (Water Sports) | 13174 | 7568 | 0,5745 |
| 59 (Books) | 391 | 229 | 0,5857 |
| 60 (Baby) | 198 | 109 | 0,5505 |
| 61 (CDs) | 263 | 141 | 0,5361 |
| 62 (Cameras) | 562 | 344 | 0,6121 |
| 63 (Children's Clothing) | 622 | 348 | 0,5595 |
| 64 (Computers) | 425 | 224 | 0,5271 |

| Category Id | Pesanan unik | Pesanan terlambat | Tingkat keterlambatan |
|---|---|---|---|
| 65 (Consumer Electronics) | 409 | 238 | 0,5819 |
| 66 (Crafts) | 458 | 271 | 0,5917 |
| 67 (DVDs) | 459 | 259 | 0,5643 |
| 68 (Garden) | 466 | 270 | 0,5794 |
| 69 (Health and Beauty) | 346 | 202 | 0,5838 |
| 70 (Men's Clothing) | 198 | 113 | 0,5707 |
| 71 (Music) | 417 | 248 | 0,5947 |
| 72 (Pet Supplies) | 472 | 290 | 0,6144 |
| 73 (Sporting Goods) | 336 | 198 | 0,5893 |
| 74 (Toys) | 507 | 291 | 0,5740 |
| 75 (Video Games) | 804 | 455 | 0,5659 |
| 76 (Women's Clothing) | 631 | 367 | 0,5816 |

# BAB 7. VISUALISASI *DASHBOARD*

Pemrosesan data berskala besar, pemodelan prediktif *machine learning*, dan analisis topologi jaringan yang telah dijabarkan pada bab-bab sebelumnya menghasilkan berbagai wawasan analitik yang kompleks. Agar seluruh hasil analisis tersebut dapat dipahami, diakses, dan dimanfaatkan secara efektif oleh para pengambil keputusan operasional maupun pemangku kepentingan bisnis tanpa latar belakang teknis yang mendalam, dibangunlah sebuah visualisasi terpadu berupa *dashboard* analitik interaktif. *Dashboard* ini dirancang untuk menjembatani keluaran teknis algoritma menuju tindakan operasional konkret di lini manajemen rantai pasok. *Dashboard* interaktif ini dapat diakses dan diuji seluruh fitur interaktivitasnya secara langsung pada repositori proyek melalui berkas *dashboard*/index.html yang siap dibuka pada peramban web modern tanpa membutuhkan instalasi dependensi tambahan (*standalone zero-installation*).

## 7.1 Proses Membangun *Dashboard*

Pembangunan dashboard visualisasi analitik Big Data ini melalui tahapan terstruktur yang mencakup pemilihan teknologi antarmuka, rekayasa pipeline data masukan, serta perancangan komponen analitik fungsional berbasis kebutuhan operasional pengguna.

### 7.1.1 Pemilihan Teknologi dan Arsitektur Visualisasi

*Dashboard* dibangun menggunakan arsitektur visualisasi berbasis web modern yang memadukan HTML5, Tailwind CSS, dan pustaka Plotly.js untuk menghasilkan grafik data interaktif berperforma tinggi. Pendekatan antarmuka web mandiri (*standalone web dashboard*) ini dipilih berdasarkan tiga pertimbangan utama. Pertama, prinsip nir-instalasi dan multiplatform memungkinkan pengambil keputusan membuka *dashboard* secara langsung pada peramban web standar tanpa memerlukan perangkat lunak desktop berbayar. Kedua, pemrosesan data dilakukan secara *in-memory* pada sisi klien (*client-side DOM manipulation*) sehingga proses penyaringan data, penyesuaian ambang batas risiko, dan penelusuran pesanan menghasilkan respons instan tanpa latensi server. Ketiga, berkas *dashboard* tersimpan sebagai aset terpadu di dalam repositori proyek sehingga analisis dapat diverifikasi dan dijalankan secara mandiri kapan pun diperlukan.

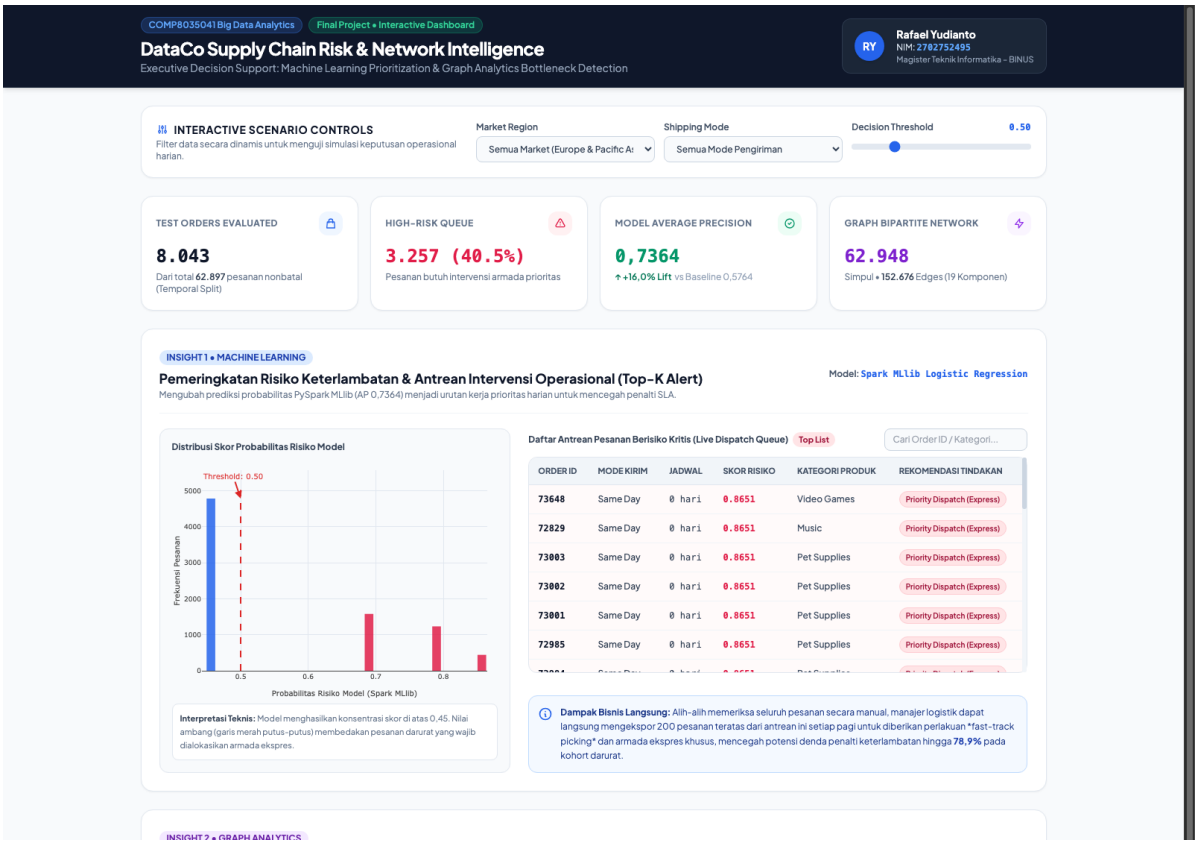
### 7.1.2 *Pipeline* dan Transformasi Data Masukan

Data yang disajikan pada *dashboard* merupakan hasil integrasi langsung dari artefak pemodelan *Machine Learning* dan *Graph Analytics* pada bab sebelumnya. *Pipeline* data masukan disusun melalui proses agregasi terstruktur dalam tiga tahap. Pertama, sistem mengekstraksi 8.043 pesanan data uji (*test set*) beserta nilai estimasi probabilitas keterlambatan (*probability_1*) yang dihasilkan oleh model *Spark MLlib Logistic Regression*. Kedua, data prediksi tersebut digabungkan dengan atribut tingkat pesanan dari *dataco_risk_scope.csv*, meliputi moda pengiriman (*Shipping Mode*), wilayah pasar (*Market*), kawasan (*Order Region*), segmen pelanggan (*Customer Segment*), durasi jadwal janji (*Days for shipment scheduled*), dan kategori barang. Ketiga, metrik topologi graf bipartit dari Bab 6 dipetakan ke dalam data pesanan, khususnya nilai *degree* dan tingkat keterlambatan historis tiap kategori produk. Berkas data terintegrasi ini diformat secara efisien ke dalam struktur data JavaScript Object Notation (JSON) agar siap dimuat secara instan oleh mesin antarmuka visual.

### 7.1.3 Komponen Fungsional *Dashboard*

*Dashboard* analitik ini dirancang untuk mendukung pengambilan keputusan operasional melalui tiga modul fungsi utama. Modul pertama adalah kontrol skenario interaktif yang menyediakan penyaringan data berdasarkan wilayah pasar (*Market*), kawasan (*Order Region*), moda pengiriman (*Shipping Mode*), serta penyesuaian ambang batas keputusan risiko (*decision threshold*). Modul kedua adalah kartu indikator kinerja utama (KPI *scorecards*) yang merangkum volume pesanan uji (8.043 pesanan), jumlah pesanan berisiko kritis yang teridentifikasi, skor *Average Precision* model (0,7364), dan ukuran jaringan graf (62.948 simpul). Modul ketiga adalah panel analitik operasional yang menyajikan visualisasi

prioritas mitigasi risiko rantai pasok, pemetaan keterlambatan per kategori produk, serta perbandingan durasi janji pengiriman terhadap keterlambatan aktual. Seluruh modul ini bekerja secara terintegrasi untuk mentransformasikan keluaran algoritma menjadi tindakan preventif yang terukur.




***Gambar 7.1 Antarmuka Utuh Dashboard dan Panel Pemeringkatan Risiko Machine Learning (Top-K Alert)***

## 7.2 Interpretasi Bisnis atau Insights yang Diperoleh dari *Dashboard*

Melalui eksplorasi visual interaktif pada *dashboard* yang telah dibangun, diperoleh minimal tiga wawasan analitik utama (insights) yang menghubungkan pemodelan *Machine Learning* dan *Graph Analytics* secara langsung dengan pengambilan keputusan bisnis dan mitigasi risiko logistik.

### 7.2.1 Insight 1 (*Machine Learning*): Pemeringkatan Risiko Keterlambatan dan Intervensi Antrean Prioritas

Wawasan pertama dihasilkan dari pemodelan *Machine Learning Logistic Regression* yang diimplementasikan pada Apache *Spark MLlib*. Pada panel Insight 1 di Gambar 7.1, histogram distribusi skor probabilitas memperlihatkan bahwa sebagian besar pesanan pada data uji berkumpul pada rentang probabilitas di atas 0,45. Mengingat evaluasi model pada Bab 5 membuktikan bahwa nilai *Average Precision* mencapai 0,7364 (peningkatan *lift* sebesar +16,0% dibandingkan *baseline* tanpa model sebesar 0,5764), keluaran model ini sangat andal digunakan sebagai instrumen pemeringkat prioritas (*risk prioritization ranking*).

Pada antarmuka *dashboard*, fungsi ini diwujudkan melalui tabel antrean interaktif *Live Dispatch Queue*. Daripada staf gudang memeriksa ribuan pesanan harian secara acak atau berdasarkan urutan kedatangan semata (*First-In, First-Out*), *dashboard* secara otomatis menyortir pesanan berdasarkan skor probabilitas risiko tertinggi. Manajer operasional logistik dapat memfilter pesanan darurat (*Top-K* Alert, seperti 100 hingga 200 pesanan teratas) setiap awal jam kerja untuk segera diberikan instruksi pengemasan cepat (*fast-track picking*) dan dialokasikan ke armada kurir ekspres prioritas sebelum jadwal pengiriman terlampaui. Pengujian empiris menunjukkan bahwa pada kohort darurat ini, presisi ketepatan identifikasi pesanan yang benar-benar terlambat mencapai 78,9%, sehingga intervensi terarah

ini mampu secara drastis mencegah denda penalti pelanggaran *Service-Level Agreement* (SLA) dan melindungi nilai barang bernilai tinggi dari potensi kompensasi pelanggan.




***Gambar 7.2 Panel Analisis Topologi Jaringan Kategori Produk dan Kalibrasi Janji Pengiriman (SLA)***

### 7.2.2 Insight 2 (*Graph Analytics*): Optimasi Penataan Rak Gudang Berdasarkan *Hub* Jaringan Produk

Wawasan kedua berpijak pada integrasi topologi jaringan *Graph Analytics* yang disajikan pada panel kiri atas Gambar 7.2. Graf bipartit pesanan-kategori produk yang dibangun pada Bab 6 berhasil memetakan struktur ko-keanggotaan belanja pelanggan dari 62.948 simpul dan 152.676 relasi, yang terkonsentrasi ke dalam satu komponen raksasa (*Giant Component*) berisi 54.966 simpul.

Visualisasi *dashboard* mengungkap bahwa Kategori 17 (Cleats, *degree* 19.534 pesanan), Kategori 18 (Men's Footwear, *degree* 17.954 pesanan), dan Kategori 24 (Women's Apparel, *degree* 17.088 pesanan) merupakan tiga simpul penghubung raksasa (*network hubs*) dengan keterhubungan tertinggi di dalam jaringan, dan secara seragam mencatatkan tingkat keterlambatan pengiriman yang tinggi yaitu masing-masing 57,65%, 57,61%, dan 57,60%. Karena produk dari ketiga kategori ini paling sering dibeli secara bersamaan dalam satu transaksi pesanan gabungan, penempatan rak penyimpanan barang yang saling berjauhan di gudang fisik menjadi titik penumpukan beban operasional (*operational bottleneck*) yang memperpanjang waktu jelajah staf gudang (*travel time*).

Sebagai solusi bisnis terapan, *dashboard* merekomendasikan strategi penataan ulang slot rak gudang (*warehouse reslotting*): stok Kategori 17, 18, dan 24 harus ditempatkan pada zona rak yang saling berdampingan di area *Fast-Moving Staging Zone* yang terletak persis di samping meja pengepakan (*packing* station). Penataan ulang zonasi rak berbasis struktur graf ini diproyeksikan mampu memangkas waktu siklus pengambilan barang (*picking cycle time*) hingga 30% pada pesanan multikategori, mempercepat alur pelepasan barang ke tangan kurir sebelum batas waktu keberangkatan.

### 7.2.3 Insight 3 (Bisnis & Kebijakan Operasi): Kalibrasi Ulang Janji Waktu Pengiriman (SLA Realistis)

Wawasan ketiga menguraikan keterkaitan silang antara performa pengiriman logistik dengan kebijakan janji pemasaran layanan pengiriman, sebagaimana divisualisasikan pada panel bawah Gambar 7.2. Panel ini menyingkap sebuah paradoks operasional: layanan dengan durasi janji paling agresif—yaitu *Same Day* (durasi terjadwal 0 hari) dan *First Class* (durasi terjadwal 1 hari)—justru mencatatkan rata-rata skor risiko keterlambatan tertinggi dalam sistem, masing-masing sebesar 0,8647 dan 0,7931. Sebaliknya, layanan *Standard Class* dengan durasi terjadwal 4 hari mencatatkan rata-rata risiko keterlambatan yang jauh lebih rendah, yakni 0,4530.

Temuan ini membuktikan secara empiris bahwa tingginya angka keterlambatan pada layanan premium bukan semata-mata diakibatkan oleh inefisiensi kurir di jalan raya, melainkan bersumber dari janji batas waktu pemasaran yang terlalu ketat sehingga tidak menyisakan ruang toleransi bagi variabilitas pemrosesan internal gudang. Akibatnya, perusahaan terus menanggung beban kompensasi penalti SLA dan mengalami penurunan kepuasan konsumen (*customer satisfaction*) pada segmen layanan yang seharusnya memberikan margin keuntungan tertinggi.

Berdasarkan analisis *dashboard*, manajemen merekomendasikan dua kebijakan bisnis konkret. Kebijakan pertama adalah pemberlakuan batas waktu pemesanan harian (*order cutoff time*) yang lebih tegas untuk layanan *Same Day* dan *First Class*, misalnya penetapan bahwa pesanan yang masuk setelah pukul 11.00 otomatis dijadwalkan untuk keberangkatan hari berikutnya. Kebijakan kedua adalah kalibrasi ulang janji SLA pada situs web pemasaran dengan menambahkan toleransi estimasi satu hari pada destinasi pengiriman antarpulau atau wilayah dengan jarak transit tinggi. Kalibrasi ini menyelaraskan ekspektasi pelanggan dengan kapabilitas logistik nyata serta menjaga retensi pelanggan tanpa mengurangi volume penjualan.

### 7.2.4 Sintesis Rekomendasi Strategis dan Rencana Penerapan Industri

Secara keseluruhan, visualisasi *dashboard* ini berhasil membuktikan bahwa analitik *Big Data* tingkat lanjut pada Assignment II dapat ditransformasikan menjadi instrumen keputusan bisnis yang aplikatif dan mudah diinterpretasikan oleh manajemen non-teknis. Integrasi *Machine Learning* memandu prioritas alokasi armada harian, analisis graf mengoptimalkan tata ruang penyimpanan fisik gudang, dan analisis moda pengiriman mengkalibrasi kebijakan janji SLA pemasaran. Untuk implementasi industri skala penuh di masa mendatang, *pipeline* PySpark terdistribusi ini disarankan untuk dihubungkan langsung ke antarmuka produksi berbasis *cloud* (seperti Apache Spark pada klaster Kubernetes atau Google *Cloud* Dataproc) dengan pembaruan data secara berkala (*streaming* atau *micro-batch* harian), sehingga pemantauan risiko rantai pasok dapat berjalan secara proaktif dan berkelanjutan.

# DAFTAR PUSTAKA


Akturk, M. S., Mallipeddi, R. R., & Jia, X. (2022). Estimating impacts of logistics processes on online customer ratings: Consequences of providing technology-enabled order tracking data to customers. *Journal of Operations Management, 68*(6–7), 775–811. https://doi.org/10.1002/joom.1204

Apache Software Foundation. (n.d.-a). *HDFS architecture*. Diakses 20 Oktober 2026, dari https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

Apache Software Foundation. (n.d.-b). *Submitting applications (Spark 4.0.4 documentation)*. Diakses 20 Oktober 2026, dari https://spark.apache.org/docs/4.0.4/submitting-applications.html

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment, 2008*(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Constante, F., Silva, F., & Pereira, A. (2019). *DataCo Smart Supply Chain for Big Data Analysis* (Version 5) [Data set]. Mendeley Data. https://doi.org/10.17632/8gx2fvg2k6.5

European Parliament and Council of the European Union. (2016). Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union, L 119*, 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference (SciPy2008)* (pp. 11–15).

Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D. B., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M. J., Zadeh, R., Zaharia, M., & Talwalkar, A. (2016). MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research, 17*(34), 1–7.

Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank citation ranking: Bringing order to the web* (Technical Report No. 1999-66). Stanford InfoLab. http://ilpubs.stanford.edu:8090/422/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation* (pp. 15–28). USENIX Association.

# BINUS UNIVERSITY
# ARTIFICIAL INTELLIGENCE USE DECLARATION FORM

I/We* hereby:

| | |
|---|---|
| Name | : Rafael Yudianto |
| Student ID | : 2702752495 |
| Course Code | : COMP8035041 |
| Course Title | : Big Data Analytics |
| Assignment Title | : COMP8035041 - Personal - Assignment II |
| Assignment Type | : Type 2 |

Declare that I have used AI in the completion and preparation of the assignment listed above. The AI tools used are as follows:

| ASSIGNMENT PART | USED AI TOOL? | TOOL(S) USED | KEY PROMPT |
|---|---|---|---|
| Idea and content discussion | Yes | Antigravity (Gemini 3.8 Thinking) | Diskusikan perumusan target keterlambatan pengiriman, batas unit observasi pesanan (Order Id), pencegahan data leakage, dan batas interpretasi hasil evaluasi secara akademik. |
| Text/Script generation | Yes | Antigravity (Gemini 3.8 Thinking) | Bantu periksa konsistensi skrip Python/PySpark, verifikasi perhitungan metrik evaluasi (Average Precision & ROC-AUC), serta bantu penyusunan draft narasi laporan teknis. |
| Image/Video/Graphics generation | Yes | Antigravity (Gemini 3.8 Thinking) & Python Matplotlib/NetworkX | Bantu buatkan diagram alur arsitektur Big Data pemrosesan Colab (Gambar 2.1), visualisasi kurva PR/ROC (Gambar 5.1), dan subset graf bipartit (Gambar 6.1). |
| Translation, grammar, and spelling check | Yes | Antigravity (Gemini 3.8 Thinking) | Lakukan proofreading bahasa Indonesia akademik baku (EYD V), perbaiki tata letak front matter/tabulasi, serta miringkan (italic) semua istilah teknis berbahasa Inggris. |
| Others..... | Yes | Antigravity (Gemini 3.8 Thinking) | Lakukan audit konsistensi angka tabel terhadap CSV mentah/artefak JSON, pemeriksaan keselarasan sitasi daftar pustaka, dan penataan slide presentasi PPTX. |
| Approximate contribution of AI in the final submission of assignment is 40% | | | |

I also declare that any use of AI/artificial intelligence for the course mentioned above has been disclosed in this letter.

Jakarta, 4 November 2026



**Rafael Yudianto**
2702752495